Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных сетей и систем
Кафедра программного обеспечения информационных технологий

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проекту
на тему

# ВЕБ-ПРИЛОЖЕНИЕ, РЕАЛИЗУЮЩЕЕ СЕТЕВУЮ ИГРУ-ГОЛОВОЛОМКУ «MIND CLASH»

БГУИР КП 6-05-0612-01 041ПЗ

Студент Кухто В. С.

Руководитель Болтак С. В.

Минск 2026

# СОДЕРЖАНИЕ

Введение ..... 6
1 Анализ предметной области и постановка задачи ..... 8
1.1 Общая характеристика сетевых игр-головоломок ..... 8
1.2 Обзор существующих решений ..... 8
1.3 Функциональные требования к системе ..... 9
2 Проектирование архитектуры приложения ..... 12
2.1 Выбор клиент-серверной модели взаимодействия ..... 12
2.2 Обоснование выбора протокола передачи данных ..... 12
3 Организация сетевого взаимодействия ..... 13
3.1 Теоретические основы передачи данных по сети ..... 13
3.2 Реализация WebSocket в Java ..... 13
3.3 Процедура установки и разрыва соединения между клиентом и сервером ..... 17
3.4 Форматы сообщений и протокол обмена ..... 19
3.5 Организация многопоточной обработки подключений ..... 20
3.6 Синхронизация состояния игры ..... 22
3.7 Обработка ошибок сети ..... 23
4 Реализация игровой логики ..... 25
5 Разработка графического интерфейса ..... 30
6 Тестирование программы ..... 34
6.1 Цели и методы тестирования ..... 34
6.2 Тестирование сетевого подключения ..... 34
6.3 Функциональное тестирование ..... 36
6.4 Анализ результатов тестирования ..... 38
7 Руководство пользователя и администратора ..... 39
7.1 Руководство пользователя ..... 39
7.2 Руководство администратора ..... 40
7.3 Настройка сервера и запуск ..... 41
Заключение ..... 43
Список использованных источников ..... 44
Приложение А (обязательное): Листинг кода с комментариями ..... 45

# ВВЕДЕНИЕ

В современном мире всё большую популярность приобретают сетевые игры, объединяющие пользователей из разных точек мира. Особый интерес представляют игры-головоломки, которые способствуют развитию логического мышления.

Целью курсового проектирования является разработка веб-приложения, реализующего сетевую игру-головоломку «Mind Clash» в соревновательном режиме для двух игроков, в тренировочном режиме для одного игрока.

Текст пояснительной записки оформляется в соответствии со стандартом предприятия «СТП 01–2024» [1]. Пояснительная записка состоит из семи разделов.

В разделе «Анализ предметной области и постановка задачи» будет проведен обзор существующих сетевых игр-головоломок, выявлены их достоинства и недостатки. На основе проведенного анализа будут сформулированы функциональные требования к разрабатываемой системе «Mind Clash».

В разделе «Проектирование архитектуры приложения» будет обоснован выбор клиент-серверной модели взаимодействия, произведен анализ и выбор оптимального протокола передачи данных, а также описана общая архитектура будущего веб-приложения.

Раздел «Организация сетевого взаимодействия» будет посвящен реализации сетевой коммуникации. Будут рассмотрены теоретические основы передачи данных, практическая реализация WebSocket-соединения на языке Java (серверная часть), описан протокол обмена сообщениями, а также вопросы многопоточной обработки подключений и синхронизации игрового состояния.

В разделе «Реализация игровой логики» будет подробно описана программная реализация правил игры, заданий и проверки результатов.

В разделе «Разработка графического интерфейса» будет представлено описание клиентской части приложения, созданной с использованием HTML5, CSS3 и JavaScript.

В разделе «Тестирование программы» будут описаны цели, методы и результаты тестирования разработанного приложения.

В разделе «Руководство пользователя и администратора» будет представлена документация, содержащая инструкции по установке, настройке серверной части и запуску приложения, а также руководство для конечных пользователей игры.

В разделе «Заключение» будут подведены итоги выполненной работы, проанализирована степень достижения поставленных целей, отмечены возможные пути дальнейшего развития и улучшения программного продукта.

В разделе «Список использованных источников» будет приведен перечень литературы и электронных ресурсов, использованных в процессе проектирования и разработки.

В «Приложении А» будет представлен листинг исходного кода программы с поясняющими комментариями.

В графической части на листе 1 будет представлена схема алгоритма работы системы [2].

# 1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ПОСТАНОВКА ЗАДАЧИ

## 1.1 Общая характеристика сетевых игр-головоломок

Сетевые игры-головоломки представляют собой игры, суть которых заключается в решении логических задач. Сетевое взаимодействие в таких играх обеспечивает соревновательный режим между игроками. Согласно данным опросов, головоломки стабильно в тройку самых популярных жанров среди пользователей [3].

Большинство современных сетевых головоломок работают прямо в браузере. Многие пользователи отмечают это как достоинство, так как не нужно скачивать обновления. Можно открыть вкладку в браузере и сразу играть.

За внешней простотой подобных игр скрывается сложный механизм синхронизации состояний игры. Каждый ход одного игрока должен мгновенно отражаться на экране другого пользователя. При этом критически важным является минимизация задержек передачи данных, обеспечение целостности игрового процесса и корректная обработка одновременных подключений.

## 1.2 Обзор существующих решений

На данный момент существует несколько аналогов разрабатываемой программы.

1 Сетевая игра-головоломка «Brave Escape».

«Brave Escape» – это головоломка, в которой два игрока должны объединить усилия, чтобы выбраться из опасной ситуации. Игра предлагает задачи, которые требуют слаженной работы, быстрой реакции и внимательности. В «Brave Escape» игроки оказываются в лабиринте, полном препятствий, загадок и ловушек, и им нужно использовать свои умения, чтобы преодолеть каждый уровень и найти выход [4].

На рисунке 1.1 можно увидеть, как выглядит один из уровней этой игры.

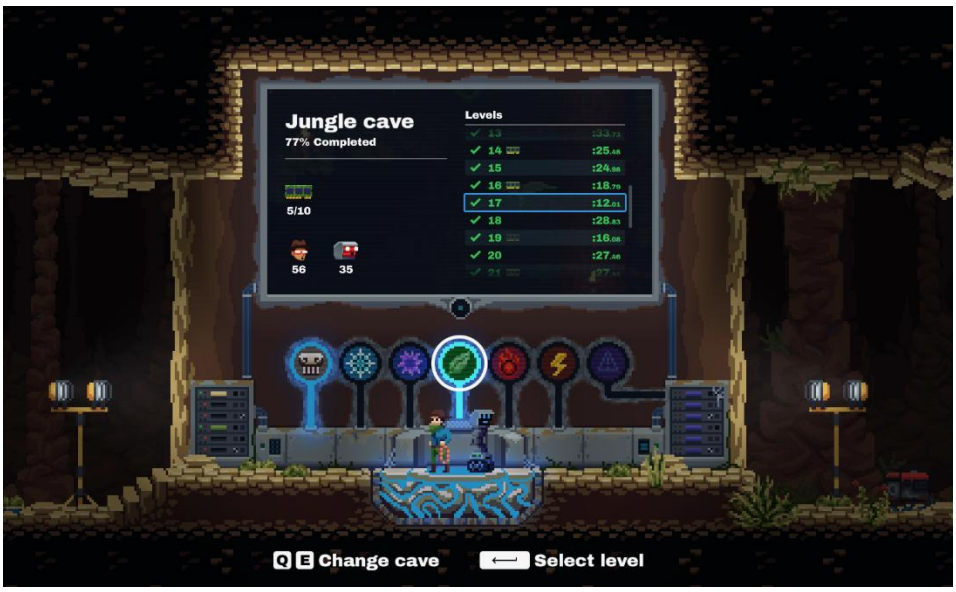


Рисунок 1.1 – один из уровней игры «Brave Escape»

2 Сетевая игра-головоломка «Escape Simulator».

«Escape Simulator» – это головоломка, которая позволяет игрокам почувствовать себя в роли участников классических квестов в стиле "escape room". Игрокам предстоит решать головоломки, искать скрытые предметы и взаимодействовать с окружением, чтобы найти выход из множества комнат. Игра поддерживает кооперативный режим [4].

На рисунке 1.2 можно увидеть, как выглядит одна из комнат этой игры.

Рисунок 1.2 – одна из игровых комнат игры «Escape Simulator»

### 1.3 Функциональные требования к системе

В данном разделе описаны основные функциональные требования разрабатываемого веб-приложения, реализующего сетевую игру-головоломку «Mind Clash». Требования сгруппированы по модулям системы.

1 Модуль регистрации и авторизации.

Система должна предоставлять пользователю возможность создания новой учетной записи. Для этого пользователю необходимо указать уникальное имя (логин), пароль, а также выбрать один из доступных аватаров профиля. При регистрации система должна проверять уникальность логина. Пароль пользователя не должен храниться в открытом виде. Перед сохранением в базе данных он должен быть захеширован с использованием алгоритма SHA-256 [5]. Для доступа к уже созданной учетной записи пользователь должен иметь возможность войти в систему, указав свой логин и пароль. Система должна проверять соответствие введенных данных и, при успешной проверке, устанавливать статус пользователя «онлайн».

2 Модуль управления профилем.

Система должна обеспечивать хранение и отображение статистики пользователя. В профиле пользователя должны отображаться его имя, выбранная иконка профиля и общее количество кубков, заработанных за победы в игровых сессиях. Количество кубков должно увеличиваться каждый раз, когда пользователь побеждает в новой игре. Игрок может редактировать свой профиль, изменив имя, пароль, логин либо аватар.

3 Модуль игровых уровней.

Система должна поддерживать три уровня сложности. При формировании игровой сессии система должна осуществлять случайную выборку заданий, соответствующую выбранной игроком сложности. Количество уровней в одной игровой сессии фиксировано и равно десяти.

4 Модуль игровой сессии

Система должна обеспечивать создание и управление игровыми сессиями для двух игроков в соревновательном режиме либо одиночном тренировочном режиме (сыграть так может любой пользователь, не прошедший регистрацию на сайте). При выборе режима «пригласить друга» система должна предоставлять поле для ввода ника друга. Система должна проверять, находится ли указанный пользователь в сети и отправлять ему приглашение присоединиться к игре. Приглашенный пользователь должен иметь возможность принять или отклонить приглашение. После того как оба игрока нажали кнопку начала игры, система должна создать игровую комнату (сессию), связать с ней обоих игроков и отправить им данные для старта (набор уровней).

5 Модуль игрового процесса. В процессе игры система должна предоставлять игрокам задания. Игроки отправляют свои ответы на сервер. Система должна проверять правильность ответа и, в случае успеха, обновлять прогресс игрока. Информация о прогрессе каждого игрока (количество пройденных уровней) должна транслироваться обоим участникам сессии в реальном времени. Система должна определять, какой из игроков побеждает. В этот момент система должна отправить обоим клиентам сообщение об окончании игры с указанием победителя. На экране победителя должно отобразиться изображение кубка.

6 Модуль сетевого взаимодействия

Система должна корректно обрабатывать подключение новых клиентов, разрыв соединения (например, при закрытии вкладки браузера), а также информировать второго игрока об отключении соперника. Для авторизации и начальной загрузки статических страниц используется протокол HTTP, реализованный отдельным модулем.

7 Модуль клиентского интерфейса

Клиентская часть, исполняемая в браузере, должна обеспечивать удобный и интуитивно понятный интерфейс. Система должна отображать стартовое окно с названием игры, режим игры с другом либо же тренировочный режим. Интерфейс должен содержать формы для регистрации и входа. На экране выбора уровня и режима должны присутствовать кнопки для выбора сложности (1, 2 или 3) и поле для ввода ника друга. Игровое поле должно отображать текущий уровень и прогресс обоих игроков в наглядном виде. При победе одного из игроков должен быть показан кубок.

# 2 ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ ПРИЛОЖЕНИЯ

## 2.1 Выбор клиент-серверной модели взаимодействия

Для веб-приложения, реализующего сетевую игру-головоломку «Mind Clash», выбрана клиент-серверная архитектура.

Клиент-серверная архитектура представляет собой модель взаимодействия, в которой участвуют три ключевых компонента: клиент, сервер и база данных. Клиент – программа, с которой пользователь работает напрямую. Сервер – компьютер, принимающий запросы, обрабатывающий их и отправляющий ответы. База данных – хранилище, где хранится вся информация: пользователи, игровые уровни, статистика и заработанные кубки.

Клиентом в разрабатываемом приложении выступает веб-браузер, который загружает HTML-страницы, CSS-стили и JavaScript-код. Вся игровая логика, проверка ответов, управление сессиями и работа с базой данных сосредоточены на серверной стороне.

## 2.2 Обоснование выбора протокола передачи данных

В разрабатываемом приложении используются следующие протоколы: HTTP для начальной загрузки страниц и авторизации и WebSocket для обеспечения игрового процесса в реальном времени.

HTTP (HyperText Transfer Protocol) – это протокол прикладного уровня, работающий по модели «запрос-ответ». Клиент отправляет запрос, сервер обрабатывает его и отправляет ответ, после чего соединение обычно закрывается [6]. В курсовом проекте HTTP используется для отдачи HTML-страниц (главная страница, формы регистрации и входа), а также для обработки первоначальной авторизации пользователей.

Для игрового процесса, где требуется постоянный обмен данными между игроками и сервером в реальном времени, протокол HTTP не подходит. Он имеет следующие недостатки: невозможность для сервера инициировать отправку данных самостоятельно, задержки, связанные с постоянным установлением новых соединений.

WebSocket обеспечивает постоянное двунаправленное соединение между клиентом и сервером поверх одного TCP-соединения.

После установки соединения как клиент, так и сервер могут отправлять сообщения независимо друг от друга в любой момент времени. Данные передаются в виде фреймов, что обеспечивает высокую эффективность и низкую задержку. Когда один игрок отправляет ответ на задание, сервер мгновенно обновляет прогресс и транслирует это изменение второму игроку.

# 3 ОРГАНИЗАЦИЯ СЕТЕВОГО ВЗАИМОДЕЙСТВИЯ

## 3.1 Теоретические основы передачи данных по сети

Передача данных в компьютерных сетях базируется на семиуровневой модели взаимодействия открытых систем (OSI). Данная модель разделяет процесс передачи на семь уровней, каждый из которых решает свои задачи и предоставляет сервисы вышестоящему уровню. В разрабатываемом приложении используются протоколы, соответствующие прикладному (HTTP, WebSocket) и транспортному (TCP) уровням.

На транспортном уровне используется протокол TCP (Transmission Control Protocol), который обеспечивает надежную передачу данных с установлением соединения. Перед началом обмена данными TCP выполняет процедуру трехстороннего рукопожатия (three-way handshake). Клиент отправляет серверу сегмент с флагом SYN, сервер отвечает сегментом SYN-ACK, и клиент завершает установку соединения сегментом ACK. После этого устанавливается канал, по которому данные передаются в виде сегментов с гарантией доставки, контролем целостности и упорядочиванием пакетов. TCP обеспечивает управление потоком данных с помощью скользящего окна, что позволяет избежать перегрузки сети и эффективно использовать доступную пропускную способность.

На прикладном уровне используются два протокола: HTTP для передачи гипертекста и WebSocket для двунаправленного обмена сообщениями в реальном времени. Протокол HTTP (HyperText Transfer Protocol) является протоколом типа «запрос-ответ». Сервер не может инициировать отправку данных клиенту, что делает HTTP непригодным для приложений, требующих реального времени.

Протокол WebSocket был разработан для преодоления ограничений HTTP. WebSocket обеспечивает полнодуплексную связь поверх одного TCP-соединения. После первоначального установления соединения через HTTP-рукопожатие, протокол переключается на WebSocket, и обе стороны могут отправлять данные независимо друг от друга[7].

## 3.2 Реализация WebSocket в Java

В курсовом проекте реализация WebSocket-сервера была выполнена без использования сторонних библиотек.

Фаза рукопожатия начинается с того, что клиент отправляет HTTP-запрос с особыми заголовками, указывающими на желание переключить протокол. На рисунке 3.1 представлен захват WebSocket handshake-запроса в программе Wireshark. Клиентский запрос имеет следующую структуру: GET / HTTP/1.1 – метод и путь, Host: localhost:8081 – хост и порт сервера, Upgrade: websocket – указание на необходимость смены протокола, Connection: Upgrade – тип соединения, Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ== – 16-байтовое значение, закодированное в Base64, Sec-WebSocket-Version: 13 – версия протокола WebSocket.

```
147 5.918387      ::1                ::1                HTTP      615 GET / HTTP/1.1

Null/Loopback
Internet Protocol Version 6, Src: ::1, Dst: ::1
Transmission Control Protocol, Src Port: 52904, Dst Port: 8081, Seq: 1, Ack: 1, Len: 551
Hypertext Transfer Protocol
  GET / HTTP/1.1\r\n
  Host: localhost:8081\r\n
  Connection: Upgrade\r\n
  Pragma: no-cache\r\n
  Cache-Control: no-cache\r\n
  User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
  Upgrade: websocket\r\n
  Origin: http://localhost:8080\r\n
  Sec-WebSocket-Version: 13\r\n
  Accept-Encoding: gzip, deflate, br, zstd\r\n
  Accept-Language: ru-BY,ru;q=0.9,be-BY;q=0.8,be;q=0.7,ru-RU;q=0.6,en-US;q=0.5,en;q=0.4\r\n
  Sec-WebSocket-Key: DNcGmdrei0uxBIyQW4FWtQ==\r\n
  Sec-WebSocket-Extensions: permessage-deflate; client_max_window_bits\r\n
```

Рисунок 3.1 – WebSocket handshake

В классе WebSocketHandler метод performHandshake() реализует чтение этого запроса. Сервер извлекает значение заголовка Sec-WebSocket-Key, конкатенирует его с фиксированным глобально-уникальным идентификатором GUID "258EAFA5-E914-47DA-95CA-C5AB0DC85B11", который определен в спецификации протокола RFC 6455. Полученная строка хешируется алгоритмом SHA-1, и результат снова кодируется в Base64. Именно это значение отправляется клиенту в заголовке Sec-WebSocket-Accept. Сервер формирует ответ, представленный на рисунке 3.2. HTTP/1.1 101 Switching Protocols – статус переключения протоколов, Upgrade: websocket – подтверждение смены протокола, Connection: Upgrade – тип соединения, Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+xOo= – вычисленный ключ.

```
149 5.919173      ::1                ::1                HTTP      193 HTTP/1.1 101 Switching Protocols

ame 149: Packet, 193 bytes on wire (1544 bits), 193 bytes captured (1544 bits) on interface \Devi
ll/Loopback
ternet Protocol Version 6, Src: ::1, Dst: ::1
ansmission Control Protocol, Src Port: 8081, Dst Port: 52904, Seq: 1, Ack: 552, Len: 129
pertext Transfer Protocol
  HTTP/1.1 101 Switching Protocols\r\n
  Upgrade: websocket\r\n
  Connection: Upgrade\r\n
  Sec-WebSocket-Accept: Nc3t3j4aPAJfpa212S2LbSz5mOI=\r\n
  \r\n
  [Request in frame: 147]
  [Time since request: 786.000 microseconds]
  [Request URI: /]
  [Full request URI: http://localhost:8081/]

0000  18
0010  00
0020  00
0030  c2
0040  48
0050  74
0060  0d
0070  63
0080  3a
0090  65
00a0  20
00b0  31
00c0  0a
```

Рисунок 3.2 – ответ сервера о смене протоколов

После этого клиент проверяет корректность вычисленного значения, и если все верно, соединение переключается на протокол WebSocket.

Фаза обмена данными осуществляется с помощью специальной структуры WebSocket-фрейма, схема которого представлена на рисунке 3.3. Класс WebSocketFrame содержит статические методы для кодирования и декодирования фреймов. Структура фрейма включает следующие поля:

1 Первый байт: старший бит – флаг FIN (Final), указывающий на то, является ли данный фрейм последним фрагментом сообщения; следующие три бита зарезервированы и должны быть равны нулю (для дальнейшего расширения протокола); младшие четыре бита содержат opcode, определяющий тип фрейма: 0x1 для текстовых данных, 0x8 для закрытия соединения, 0x9 для ping и 0xA для pong.

2 Второй байт: старший бит – флаг MASK (маскирование), указывающий, замаскированы ли данные; младшие семь бит содержат длину полезной нагрузки. Если длина не превышает 125 байт, она хранится непосредственно во втором байте. Если длина составляет от 126 до 65535 байт, во втором байте устанавливается значение 126, а следующие два байта содержат фактическую длину. Для сообщений длиннее 65535 байт во втором байте устанавливается 127, и следующие восемь байт содержат длину.

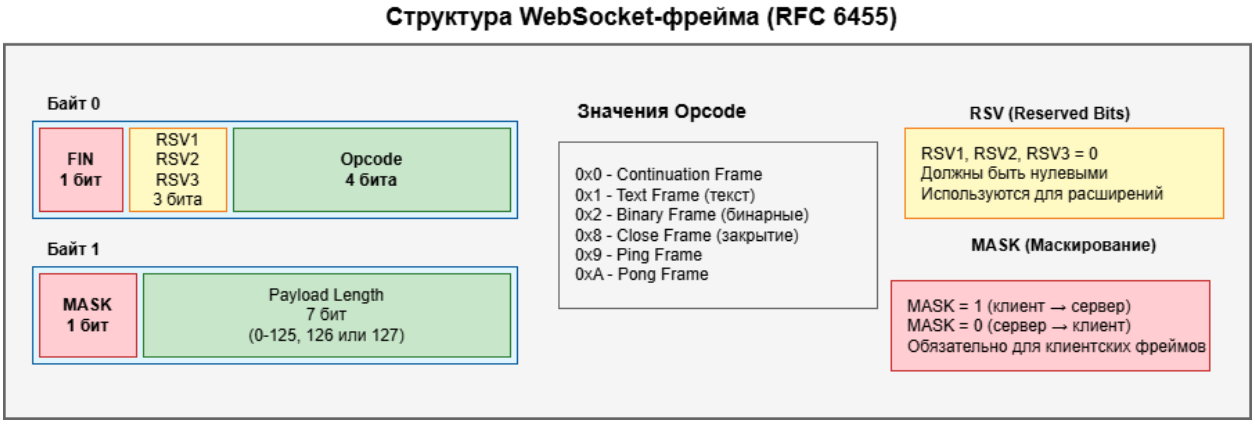


Рисунок 3.3 – Структура WebSocket-фрейма

На рисунке 3.4 представлен захват WebSocket-фрейма в Wireshark, где можно увидеть структуру передаваемых данных. Как видно в первом выделенном пакете, данные замаскированы, потому что передаются от клиента к серверу (на скриншоте порт назначения 8081). Клиент передает фрейм с содержимым о том, что пользователь хочет авторизоваться.

```
websocket
No.  Time        Source     Destination  Protocol  Length  Info
851  10.354598   127.0.0.1  127.0.0.1    WebSoc…   103     WebSocket Text [FIN] [MASKED]
907  10.374428   127.0.0.1  127.0.0.1    WebSoc…   113     WebSocket Text [FIN]

> Frame 851: Packet, 103 bytes on wire (824 bits), 103 bytes captured (824 bits) on interface \Device\NPF_Lo
> Null/Loopback
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
> Transmission Control Protocol, Src Port: 50551, Dst Port: 8081, Seq: 554, Ack: 130, Len: 59
v WebSocket
    1... .... = Fin: True
    .000 .... = Reserved: 0x0
    .... 0001 = Opcode: Text (1)
    1... .... = Mask: True
    .011 0101 = Payload length: 53
    Masking-Key: 28a5e644
    Masked payload
  > Payload
v Line-based text data (1 lines)
    {"type":"login","username":"vera","password":"12345"}
```

Рисунок 3.4 – Пример Web-socket фреймов

Особенностью протокола WebSocket является требование маскирования всех данных, отправляемых от клиента к серверу. Это требование введено для предотвращения различных атак. Маскирование реализуется с помощью 32-битного ключа, который клиент генерирует случайным образом и отправляет в составе фрейма. Для каждого байта полезной нагрузки применяется операция XOR с соответствующим байтом ключа маскирования, где ключ циклически повторяется. В методе decodeTextFrame класса WebSocketFrame реализовано чтение четырехбайтового ключа маскирования и последующее применение XOR для расшифровки полезной нагрузки (рисунок 3.5). Сервер, в свою очередь, не должен маскировать отправляемые данные, поэтому в методе encodeTextFrame для фреймов, отправляемых сервером, флаг MASK всегда устанавливается в 0.

```
if (masked) {
    for (int i = 0; i < payload.length; i++) {
        payload[i] = (byte) (payload[i] ^ maskKey[i % 4])
    }
}
```

Рисунок 3.5 – Часть кода для расшифровки фрейма с помощью операции XOR

После завершения обмена данными или при возникновении ошибки происходит закрытие соединения. Для корректного завершения используется специальный фрейм закрытия с opcode 0x8 (рисунок 3.6), который может содержать код ошибки и текстовое описание причины закрытия. В разработанной

системе закрытие соединения обрабатывается в методе close() класса WebSocketHandler, который устанавливает флаг running в false, удаляет клиента из реестра подключений с помощью метода removeClient, и закрывает сокет.

```
483 31.151050   127.0.0.1   127.0.0.1   WebSoc…   52 WebSocket Connection Close [FIN] [MASKED]

Frame 483: Packet, 52 bytes on wire (416 bits), 52 bytes captured (416 bits) on interface \Device\NPF_Loopback, id 0
Null/Loopback
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
Transmission Control Protocol, Src Port: 60963, Dst Port: 8081, Seq: 613, Ack: 199, Len: 8
WebSocket
    1... .... = Fin: True
    .000 .... = Reserved: 0x0
    .... 1000 = Opcode: Connection Close (8)
    1... .... = Mask: True
    .000 0010 = Payload length: 2
    Masking-Key: 36d7561c
    Masked payload
```

Рисунок 3.6 – Запрос клиента к серверу на завершение соединения с opcode 0x8

### 3.3 Процедура установки и разрыва соединения между клиентом и сервером

Когда клиент (веб-браузер) инициирует подключение к серверу, операционная система выполняет TCP-рукопожатие (рисунок 3.4, скриншот из программы WireShark), описанное в разделе 3.7.

```
tcp.port==8080
No.  Time        Source   Destination  Protocol  Length  Info
70   2.911675    ::1      ::1          TCP       76      56579 → 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=65475
71   2.911765    ::1      ::1          TCP       76      8080 → 56579 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=
72   2.911812    ::1      ::1          TCP       64      56579 → 8080 [ACK] Seq=1 Ack=1 Win=65280 Len=0
```

Рисунок 3.7 – TCP-рукопожатие

Клиент → Сервер: SYN (Seq=0, Win=65535) – клиент инициирует соединение. Сервер → Клиент: SYN, ACK (Seq=0, Ack=1, Win=8192) – сервер подтверждает и отправляет свой SYN. Клиент → Сервер: ACK (Seq=1, Ack=1, Win=65535) – клиент подтверждает получение.

После установления TCP-соединения серверный поток, ожидающий в методе accept() на ServerSocket, получает новый объект Socket для этого клиента. Сервер немедленно передает обработку этого сокета в отдельный поток из пула ExecutorService, что позволяет основному потоку продолжать принимать новые подключения. В рамках обработчика ClientHandler сервер читает HTTP-запрос построчно с помощью BufferedReader, извлекает метод и путь запроса, формирует соответствующий ответ и отправляет его обратно клиенту (рисунок 3.8, скриншот из программы WireShark, пакеты соответствующие http-запросу и ответу при авторизации зарегистрированного пользователя). После отправки ответа соединение закрывается.

| | | | | | |
|---|---|---|---|---|---|
| 33 | 8.555778 | ::1 | ::1 | HTTP | 991 POST /login HTTP/1.1 (application/x-www-form-urlencoded) |
| 49 | 8.764666 | ::1 | ::1 | HTTP | 493 HTTP/1.1 200 OK (text/html) |

Рисунок 3.8 – http-запрос и ответ

Процедура установки WebSocket-соединения на порту 8081: сначала стандартное HTTP-соединение для выполнения рукопожатия, а затем переход на WebSocket-протокол с сохранением открытого сокета. На первой фазе, как описано в разделе 3.2, клиент отправляет HTTP-запрос с заголовками Upgrade. Сервер, получив этот запрос, не закрывает соединение после отправки ответа 101 Switching Protocols, а оставляет сокет открытым для дальнейшего обмена WebSocket-фреймами.

Ключевое отличие заключается в том, что в отличие от HTTP, где каждый запрос обрабатывается независимо, WebSocket-соединение остается активным на протяжении всей игровой сессии, что позволяет серверу отправлять данные клиенту в любой момент без ожидания запроса.

Механизм управления пулом подключений реализован в классе WebSocketServer с использованием коллекции ConcurrentHashMap, которая хранит соответствие между идентификатором пользователя и его WebSocketHandler. Когда клиент успешно проходит авторизацию через WebSocket, вызывается метод registerClient, который добавляет запись в эту коллекцию. Благодаря использованию ConcurrentHashMap, несколько потоков могут одновременно добавлять и удалять клиентов без риска возникновения ConcurrentModificationException. При разрыве соединения, будь то инициатива клиента или потеря сетевого соединения, вызывается метод removeClient, который удаляет запись из коллекции и уведомляет других игроков о разрыве соединения.

Разрыв соединения может происходить по нескольким причинам. Нормальное завершение инициируется клиентом с помощью отправки WebSocket-фрейма закрытия (opcode 0x8), соответствующий скриншот был в разделе 3.2 рисунок 3.6, после чего сервер отвечает аналогичным фреймом и закрывает сокет. Аварийное завершение происходит при обнаружении IOException в потоке чтения или записи, что свидетельствует о потере сетевого соединения. В этом случае метод handleMessage ловит исключение и переходит к блоку finally, где вызывается закрытие. Важно отметить, что при отключении игрока во время активной игры, сервер уведомляет соперника с помощью сообщения типа "opponent_disconnected" и помечает игровую комнату как неактивную, предотвращая дальнейшие попытки отправки ответов от отключившегося игрока, на рисунке 3.9 можно увидеть соответствующее сообщение в консоли разработчика в браузере. В классе GameRoom для этого используется метод setPlayerDisconnected, который устанавливает флаг connected в false и деактивирует игру.

```
Получено:                                                    websocket-client.js:22
{"type":"opponent_disconnected","message":"Соперник отключился"}
>
```

Рисунок 3.9 – Сообщение об отключении соперника в консоли разработчика

### 3.4 Форматы сообщений и протокол обмена

В игре используются два различных формата сообщений, соответствующие используемым протоколам: HTTP для начальной загрузки страниц и аутентификации и JSON для отправки сообщений.

Формат HTTP-сообщений используется для передачи статических файлов и обработки форм регистрации и входа. HTTP-запрос состоит из стартовой строки, заголовков и тела. Стартовая строка содержит метод (GET или POST), путь и версию протокола. Заголовки представляют собой пары "ключ: значение", разделенные \r\n, причем пустая строка служит разделителем между заголовками и телом запроса. В методе handlePost класса ClientHandler реализован парсинг заголовка Content-Length, который указывает размер тела запроса. После чтения всех заголовков до пустой строки, сервер считывает ровно contentLength символов в массив char, что позволяет корректно обрабатывать тела запросов, содержащие специальные символы. Тело POST-запроса имеет формат application/x-www-form-urlencoded, где параметры разделены символом &, а ключ и значение – знаком =. Например, "username=vera&password=123". Для декодирования используется метод parseUrlEncoded, который применяет URLDecoder для корректной обработки специальных символов, таких как пробелы или кириллица.

HTTP-ответ формируется классом HttpResponseBuilder, который собирает ответ по частям: статусная строка (например, "HTTP/1.1 200 OK"), заголовки с указанием Content-Type и Content-Length, пустая строка-разделитель, и тело ответа. Для статических файлов заголовок Content-Type определяется по расширению файла методом getMimeType: .html → text/html, .css → text/css, .js → application/javascript, .png → image/png. Для успешной регистрации или входа сервер возвращает HTML-страницу с JavaScript-кодом, который выводит уведомление и перенаправляет пользователя.

Http-ответы и запросы были представлены для примера в разделе 3.2.

Формат WebSocket-сообщений после установки соединения представляет собой JSON-объекты, передаваемые внутри текстовых фреймов WebSocket. Каждое сообщение имеет поле type, определяющее его назначение. Сообщения от клиента к серверу включают следующие типы: login – для авторизации, содержит username и password; create_room – для создания новой игровой комнаты, содержит difficulty (уровень сложности от 1 до 3); join_room – для присоединения к существующей комнате по roomId; invite – для отправки приглашения другому игроку, содержит toUsername и difficulty; accept_invite и reject_invite – для ответа на приглашение, содержат inviteId; answer – для отправки ответа на вопрос, содержит roomId и answer; ping – для поддержания соединения.

Сообщения от сервера к клиенту включают: login_success – подтверждение авторизации, содержит userId, username и trophies; login_error – ошибка авторизации; room_created – подтверждение создания комнаты, содержит roomId; room_joined – подтверждение присоединения к комнате; invite – уведомление о получении приглашения, содержит inviteId, fromUserId, fromUsername, difficulty; invite_sent – подтверждение отправки приглашения; invite_accepted – уведомление о принятии приглашения, содержит roomId и opponentId; invite_rejected – уведомление об отклонении; game_start – сигнал о начале игры; answer_result – результат проверки ответа, содержит correct (boolean), progress (текущий уровень), totalLevels; progress_update – обновление прогресса обоих игроков, содержит player1Progress и player2Progress; game_over – завершение игры, содержит winnerId и winnerUsername; opponent_disconnected – уведомление об отключении соперника; pong – ответ на ping.

На рисунке 3.10 WebSocket сообщение login (скриншот из консоли браузера с JSON-объектом).

```
Отправка login: vera                                    websocket-client.js:66
Получено:                                               websocket-client.js:22
{"type":"login_success","userId":4,"username":"vera","trophies":21}
```

Рисунок 3.10

Все сообщения формируются методом toJson в соответствующих классах-моделях или вручную с помощью конкатенации строк. На рисунке 3.11 можно увидеть метод toJson класса User для преобразования в Json перед отправкой клиенту.

```
public String toJson() {  no usages  VeronikaKukhta
    return String.format(
            "{\"id\":%d,\"username\":\"%s\",\"avatar\":\"%s\",\"trophies\":%d,\"online\":%b}",
            id, username, avatar, trophies, online
    );
}
```

Рисунок 3.11

### 3.5 Организация многопоточной обработки подключений

В разработанной системе использованы две стратегии организации многопоточности: пул потоков (ThreadPool) для обработки входящих подключений и ConcurrentHashMap для потокобезопасного хранения состояния клиентов.

При запуске каждого сервера создается пул потоков с фиксированным размером. Для HTTP-сервера используется Executors.newFixedThreadPool(10), что означает создание пула из десяти потоков, которые будут

переиспользоваться для обработки HTTP-запросов. В основном цикле while(running) сервер вызывает метод accept() на ServerSocket, который блокируется до появления нового подключения. Как только подключение установлено, сервер передает обработку этого клиента в пул потоков с помощью метода threadPool.execute(new ClientHandler(clientSocket)). При этом основной поток немедленно возвращается к accept() для ожидания следующего клиента, что обеспечивает высокую пропускную способность сервера. Аналогичным образом организована работа WebSocket-сервера, но с пулом из двадцати потоков, так как WebSocket-соединения являются долгоживущими и требуют больше ресурсов. Соответствующий код для http-сервера можно увидеть на рисунке 3.12.

```
public void start() {  2 usages  VeronikaKukhta
    try {
        serverSocket = new ServerSocket(PORT);
        running = true;
        threadPool = Executors.newFixedThreadPool( nThreads: 10);


        System.out.println("   в браузере: http://localhost:" + PORT);

        while (running) {
            Socket clientSocket = serverSocket.accept();
            threadPool.execute(new ClientHandler(clientSocket));
        }

    } catch (IOException e) {
        System.err.println(" Ошибка HTTP сервера: " + e.getMessage());
    }
}
```

Рисунок 3.12 – Создание пула потоков и ожидание подключения в коде

Каждый обработчик клиента (ClientHandler для HTTP и WebSocketHandler для WebSocket) реализует интерфейс Runnable и выполняется в отдельном потоке из пула. Внутри метода run() обработчик создает потоки ввода-вывода на основе сокета клиента и входит в цикл обработки. Для HTTP-обработчика цикл не требуется, так как после обработки одного запроса соединение закрывается. Для WebSocket-обработчика цикл продолжается до тех пор, пока не произойдет разрыв соединения или не будет вызван метод close(). Это позволяет одному потоку обслуживать одно WebSocket-соединение на протяжении всей игровой сессии, что эффективно с точки зрения использования ресурсов, так как поток блокируется на операциях чтения, не потребляя процессорное время.

Проблема синхронизации доступа к общим данным решается с помощью ConcurrentHashMap. В классе WebSocketServer определены три такие коллекции: connectedClients для хранения соответствия userId → WebSocketHandler, activeRooms для хранения соответствия roomId → GameRoom, и pendingInvites для хранения ожидающих приглашений. ConcurrentHashMap обеспечивает потокобезопасность без необходимости явной синхронизации через synchronized, что значительно упрощает код и повышает производительность. Например, когда клиент отправляет сообщение, обработчик вызывает метод sendToUser, который обращается к connectedClients.get(userId). Если бы использовалась обычная HashMap, то при одновременном добавлении и чтении из разных потоков могло бы возникнуть исключение ConcurrentModificationException или состояние гонки.

### 3.6 Синхронизация состояния игры

Синхронизация состояния игры является важной задачей, поскольку два игрока, находящиеся в разных сетевых сессиях, должны видеть согласованную картину происходящего. В разработанной системе синхронизация реализована через централизованное хранилище игровых комнат (GameRoom).

Класс GameRoom хранит все состояние игровой сессии: идентификаторы игроков, их имена, выбранный уровень сложности, список уровней, прогресс каждого игрока (количество пройденных уровней), статус подключения игроков и флаг активности игры. Все эти данные хранятся в потокобезопасных коллекциях ConcurrentHashMap, что позволяет нескольким потокам одновременно читать и модифицировать состояние комнаты без риска повреждения данных. Когда игрок отправляет ответ на вопрос, вызывается метод submitAnswer, который выполняет проверку: получает текущий прогресс игрока, проверяет правильность ответа с помощью метода checkAnswer класса Level, и если ответ верен, увеличивает счетчик прогресса.

Оба игрока проходят одни и те же уровни в одинаковом порядке. Список уровней формируется при создании комнаты методом LevelDAO.getLevelsByDifficulty, который выбирает случайные уровни заданной сложности. Поскольку список генерируется один раз и сохраняется в объекте GameRoom, оба игрока получают идентичные уровни, но проходят их независимо: каждый игрок имеет свой индекс текущего уровня (хранится в playerProgress). Это позволяет игрокам двигаться с разной скоростью, не блокируя друг друга. Количество и распределение по заданиям можно увидеть на рисунке 3.13 (фрагмент кода).

```
public static List<Level> getLevelsByDifficulty(int difficulty, int totalCount) {
    List<Level> allLevels = new ArrayList<>();



    int requiredPuzzle = 2;
    int requiredMemory = 2;
    int requiredSchulte = 2;
    int requiredStroop = 4;
```

Рисунок 3.13

Синхронизация прогресса между игроками осуществляется через механизм широковещательных уведомлений. Когда один игрок дает правильный ответ, сервер обновляет его прогресс, а затем отправляет сообщение progress_update обоим игрокам с актуальными значениями player1Progress и player2Progress. Это позволяет каждому клиенту отображать не только свой прогресс, но и прогресс соперника в реальном времени. Метод broadcastToRoom класса WebSocketServer обеспечивает отправку сообщения всем участникам комнаты, за исключением указанного пользователя (если требуется). Благодаря использованию ConcurrentHashMap для хранения клиентов, отправка сообщений выполняется быстро и безопасно.

Проверка окончания игры выполняется после каждого правильного ответа с помощью метода checkWinner, который сравнивает прогресс каждого игрока с общим количеством уровней. Как только один из игроков достигает значения levels.size(), определяется победитель, игра помечается как завершенная (gameActive = false), и всем игрокам отправляется сообщение game_over с указанием идентификатора и имени победителя. После этого игровая комната удаляется из хранилища activeRooms, освобождая ресурсы.

### 3.7 Обработка ошибок сети

На уровне установки соединения обрабатываются ошибки, связанные с невозможностью запустить сервер. В методе start классов HttpServer и WebSocketServer конструктор ServerSocket может выбросить IOException, если указанный порт уже занят или если у приложения недостаточно прав для его использования. В этом случае исключение перехватывается, выводится сообщение об ошибке, и сервер не запускается. Это предотвращает ситуацию, когда сервер работает некорректно и может привести к непредсказуемому поведению.

При обработке каждого клиентского подключения используется конструкция try-with-resources для автоматического закрытия потоков ввода-вывода. Это гарантирует, что даже в случае возникновения исключения ресурсы будут корректно освобождены. Например, в методе run класса ClientHandler создаются InputStream, OutputStream и BufferedReader, и все они

автоматически закрываются при выходе из блока try, что предотвращает утечку данных, которая могла бы привести к исчерпанию системных ресурсов при длительной работе сервера.

В процессе чтения HTTP-запроса обрабатываются ситуации, когда клиент отправляет некорректные данные. Если строка запроса не может быть распарсена (например, метод handleRequest получает parts.length < 2), сервер возвращает ответ 400 Bad Request с пояснением. При чтении заголовков, если Content-Length отсутствует в POST-запросе или не может быть преобразован в число, возникает исключение NumberFormatException, которое перехватывается и приводит к возврату ошибки клиенту. Это защищает сервер от некорректных или вредоносных запросов.

Для WebSocket-соединений обработка ошибок более сложная. В методе run класса WebSocketHandler основной цикл обрабатывает IOException, который может возникнуть при чтении данных из сокета. Это исключение обычно свидетельствует о разрыве сетевого соединения: клиент закрыл браузер, потерял интернет или был завершен принудительно. В этом случае выполнение переходит в блок finally, где вызывается метод close(), корректно завершающий работу обработчика. При этом пользователь удаляется из реестра connectedClients, и если он участвовал в игре, соперник получает уведомление об отключении (рисунок 3.14).

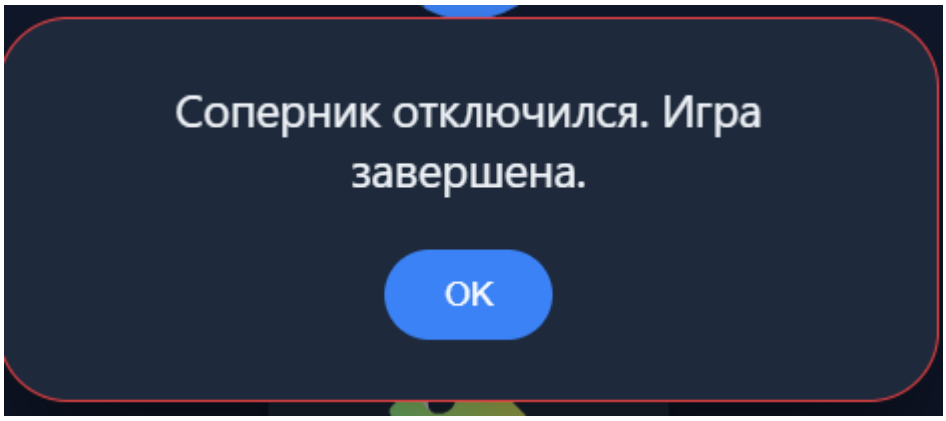


Рисунок 3.14

При декодировании WebSocket-фреймов метод decodeTextFrame может вернуть null, если фрейм имеет некорректную структуру. В этом случае соединение также закрывается, так как дальнейшая работа с клиентом, отправляющим некорректные данные, невозможна.

# 4 РЕАЛИЗАЦИЯ ИГРОВОЙ ЛОГИКИ

В основе игровой логики лежит класс GameRoom, который представляет собой виртуальное пространство для проведения одной игровой сессии между двумя участниками либо игровой сессии в тренировочном режиме для одного игрока.

В игре используются 4 типа заданий. Первый тип – текстовые задания на эрудицию. Игрок вводит ответ в поле ввода, сервер сравнивает его с правильным ответом, приведённым к нижнему регистру. Такой подход позволяет засчитывать ответы, написанные в любом регистре. На рисунке 4.1 представлен внешний вид экрана с таким заданием.

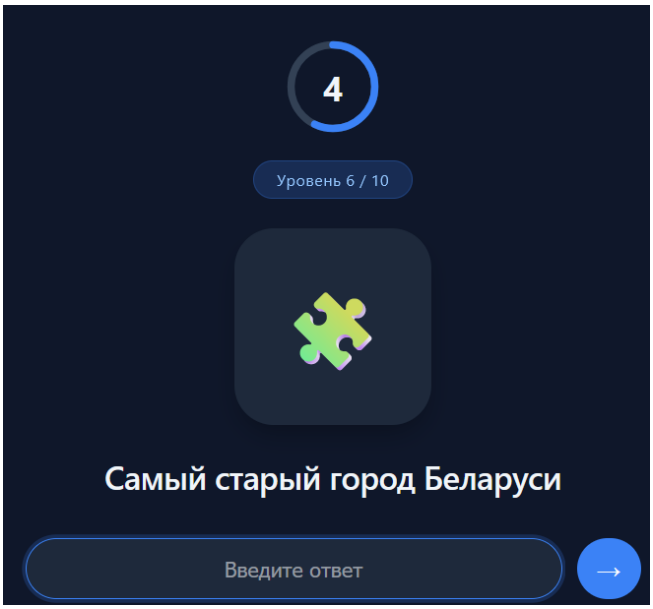


Рисунок 4.1 – Прохождение задания на эрудицию

Второй тип заданий – «Запомни число». Игроку на короткое время показывается число, которое затем скрывается, после чего требуется ввести его в поле ввода. В зависимости от уровня сложности число может содержать четыре, шесть или восемь цифр. На рисунке 4.2 показан момент, когда число ещё отображается на экране.

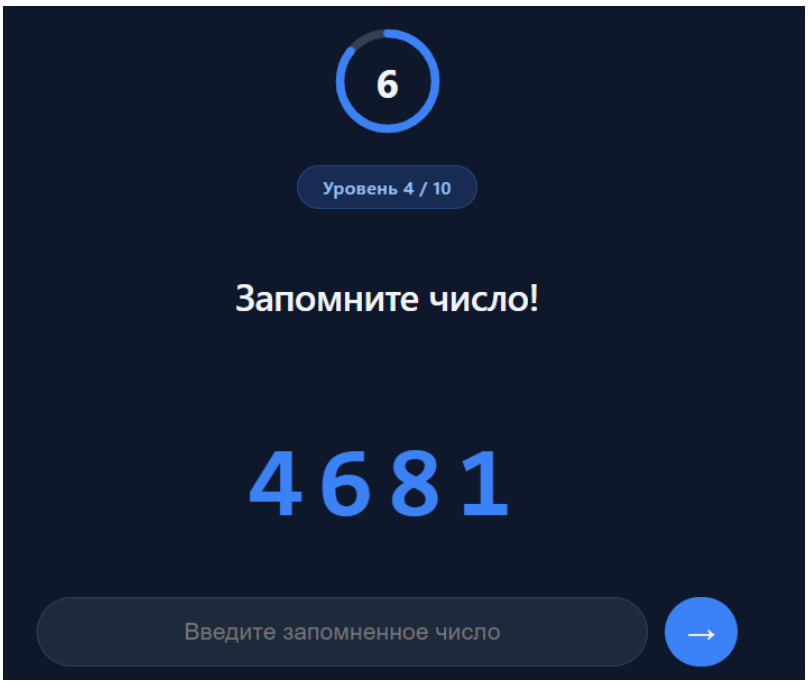


Рисунок 4.2 – Прохождение задания на запоминание числа

Третий тип заданий – таблица Шульте. На экране появляется квадратная сетка, в ячейках которой в случайном порядке расположены числа. Игрок должен последовательно нажимать на ячейки в порядке возрастания чисел. Размер сетки зависит от уровня сложности: для лёгкого уровня используется сетка два на два, для среднего – три на три, для сложного – четыре на четыре. На рисунке 4.3 показан пример таблицы Шульте для среднего уровня сложности.

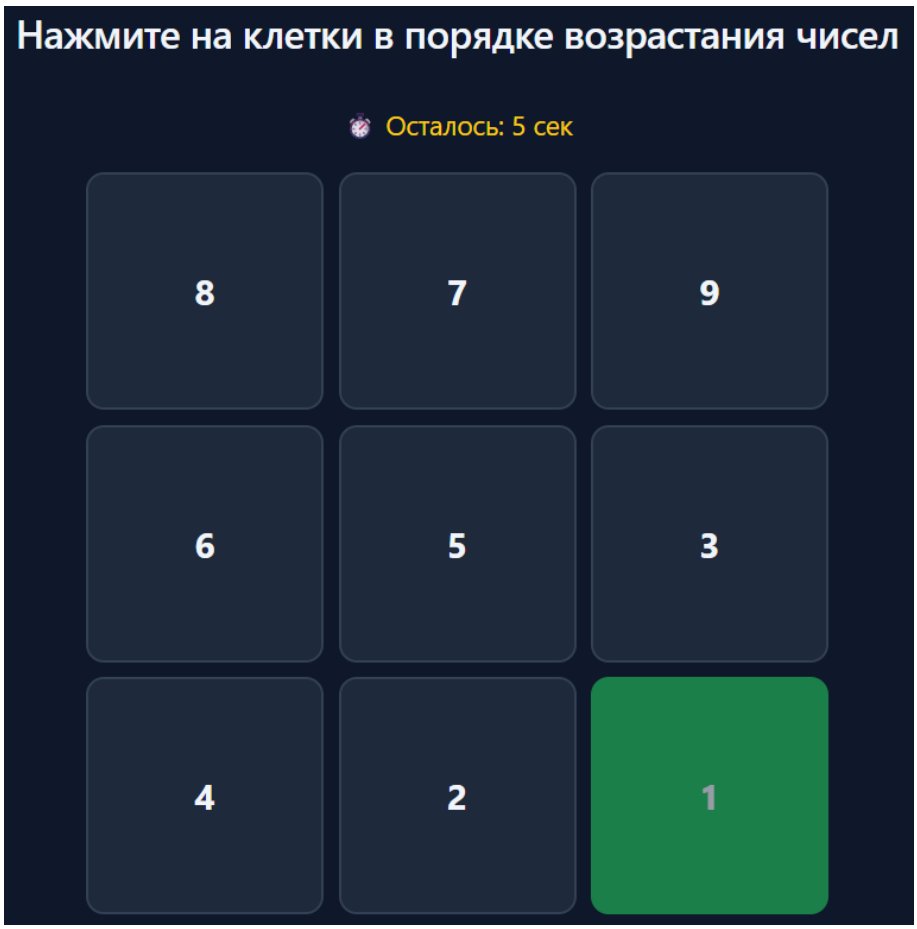


Рисунок 4.3 – Прохождение уровня с таблицей Шульте

Четвертый тип заданий – задания на эффект Струпа. На экране появляется название какого-либо цвета, однако цвет шрифта не соответствует описанию. Игрок должен должен записать в поле тот цвет, который он видит перед собой. Пример такого задания можно увидеть на рисунке 4.4. Здесь пользователю нужно ввести в ответ слово розовый, не смотря на то что перед ним написано красный.

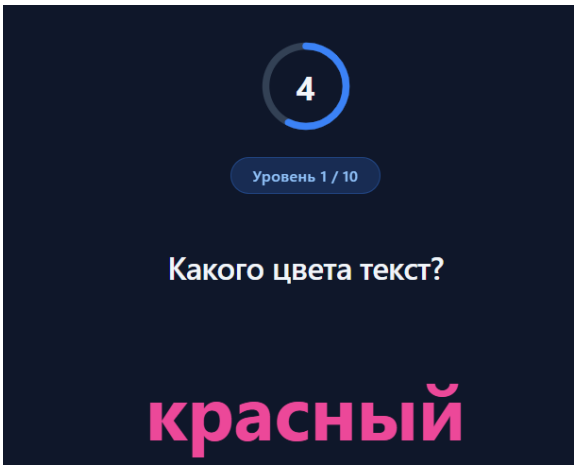


Рисунок 4.4 – Прохождение задания на эффект Струпа

Каждое задание ограничено по времени. Визуально таймер представлен в виде кругового индикатора, который постепенно заполняется. При приближении времени к нулю индикатор меняет цвет с синего на красный. На рисунке 4.5  при критическом остатке времени.

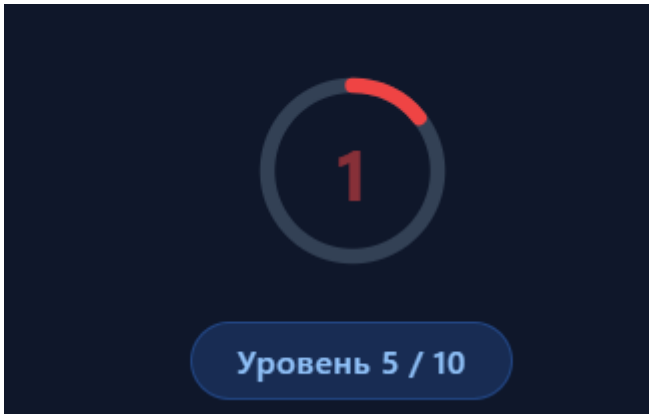


Рисунок 4.5 – Круговой таймер

Игра завершается, когда оба игрока ответили на все вопросы. Победитель определяется по количеству правильных ответов. На рисунке 4.6 показано сообщение об окончании игры с указанием победителя и числом верных ответов с обоих сторон.

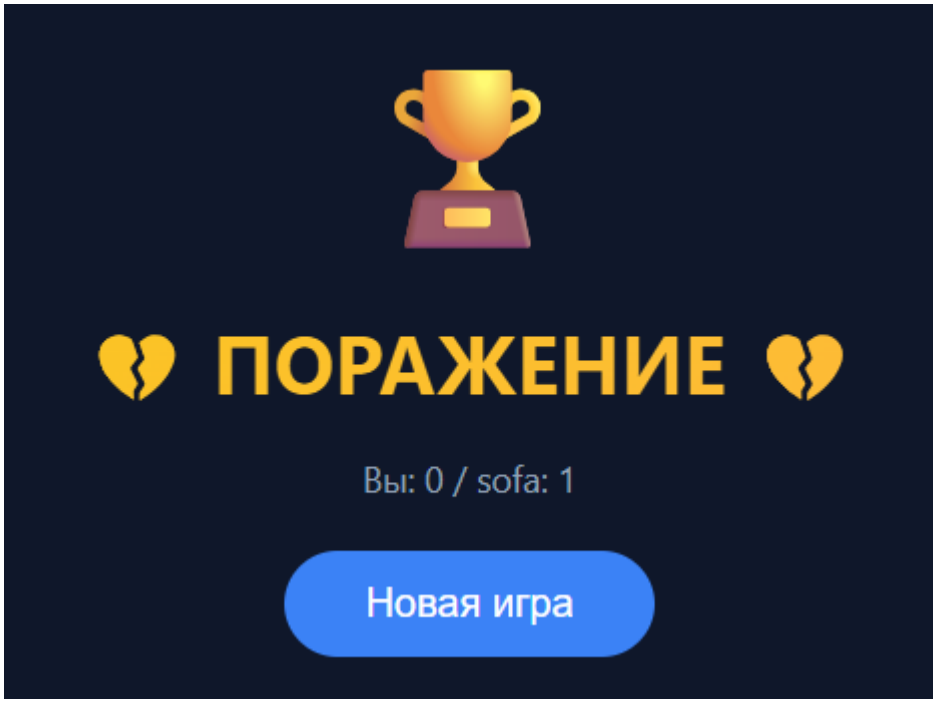


Рисунок 4.6 – Экран после завершения игры

При отключении одного из игроков во время матча второй получает соответствующее уведомление, после чего возвращается на главный экран для начала новой игры (рисунок такого уведомления был приведен в разделе 3.7).

Также любой пользователь может поиграть во все названные ранее типы игр без регистрации. Рисунок меню для тренировочного режима представлен на рисунке 4.7.

Рисунок 4.7 – Варинты одиночного режима игры

# 5 РАЗРАБОТКА ГРАФИЧЕСКОГО ИНТЕРФЕЙСА

Для создания пользовательского интерфейса использовался следующий набор веб-технологий: HTML5 для разметки страниц, CSS3 для визуального оформления и JavaScript.

Интерфейс выполнен в тёмной цветовой гамме. Фоновый градиент плавно переходит от тёмно-синего к насыщенному фиолетовому оттенку. Основные элементы оформлены в виде карточек с полупрозрачной подложкой. Все кнопки и интерактивные элементы имеют плавные анимации при наведении курсора. На рисунке 5.1 показан главный экран приложения.

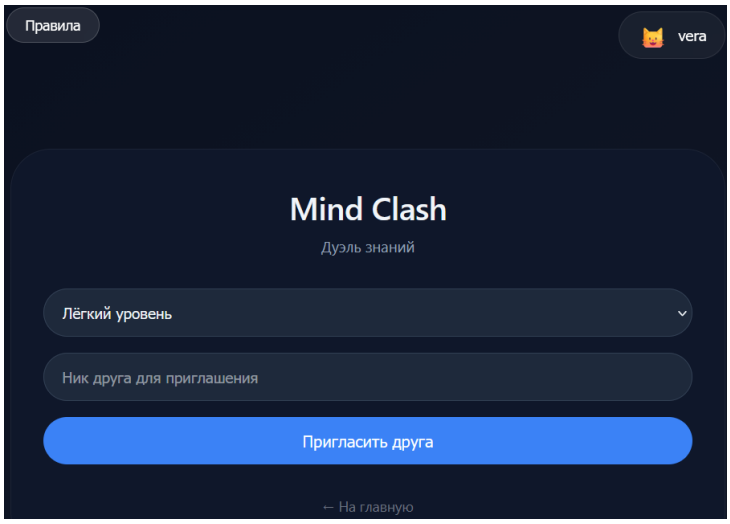


Рисунок 5.1 – Главный экран приложения

Главный экран содержит форму для выбора уровня сложности и поле для ввода ника друга, которому будет отправлено приглашение. Доступны три уровня: лёгкий, средний и сложный. Каждый уровень отличается набором заданий – чем выше уровень, тем сложнее вопросы. Пользователь вводит имя друга и нажимает кнопку приглашения, после чего сервер отправляет уведомление выбранному пользователю. Здесь же пользователь может просмотреть правила игры (рисунок 5.2) и зайти в режим редактирования профиля (рисунок 5.3).

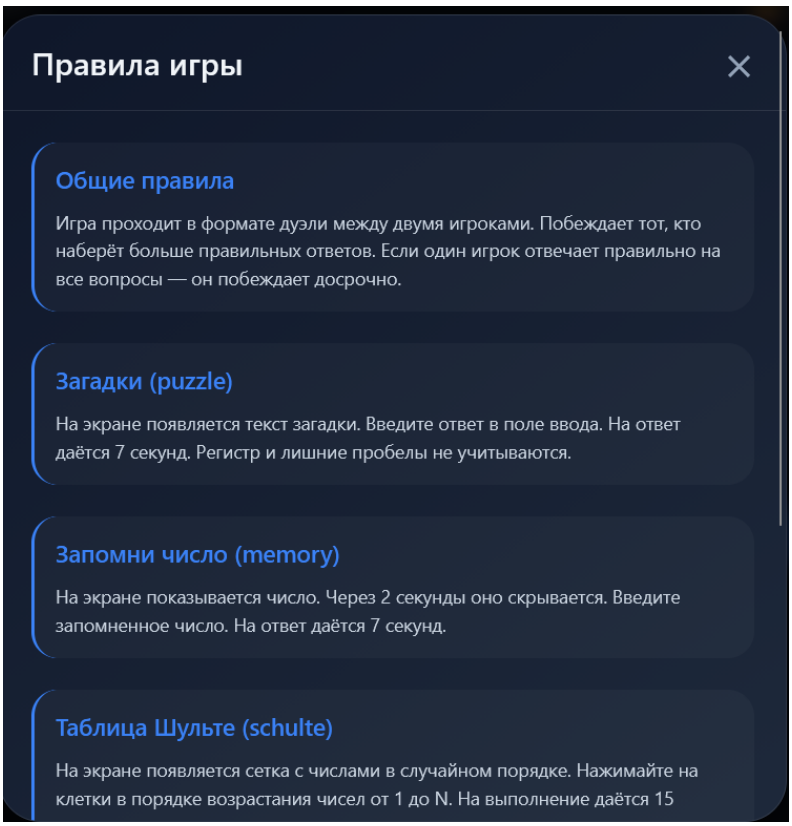


Рисунок 5.2 – Правила игры

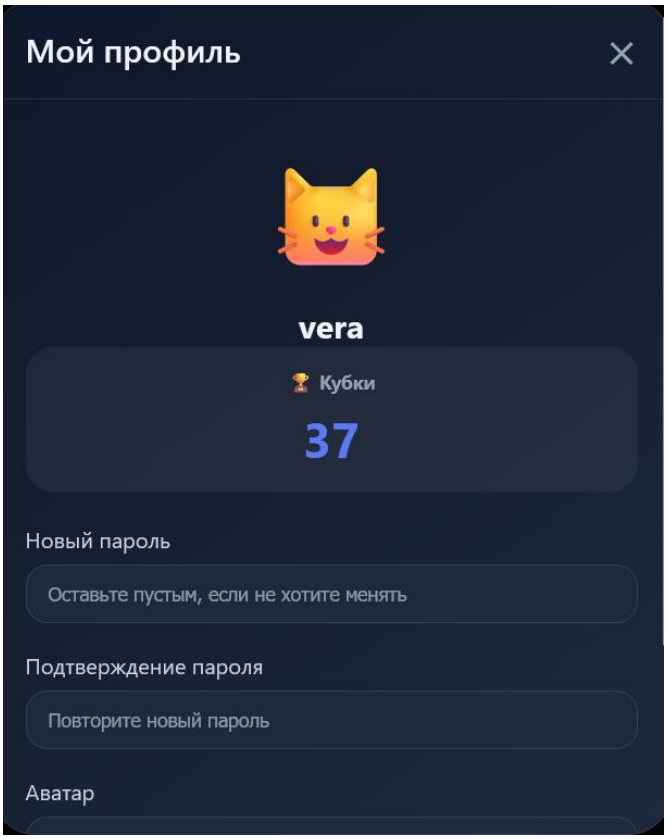


Рисунок 5.3 – Редактирование профиля

Игровая область представляет собой карточку, в которой отображается номер текущего уровня, круговой таймер, содержание задания и поле для ввода ответа. Круговой таймер реализован с использованием технологии SVG, что позволяет создавать плавную анимацию заполнения. При истечении времени таймер меняет цвет с синего на красный, привлекая внимание игрока. В зависимости от типа задания содержимое карточки динамически изменяется. Для текстовых загадок показывается вопрос и поле для ввода ответа. Для заданий на запоминание числа сначала крупным шрифтом выводится само число, которое через две секунды заменяется тремя вопросительными знаками, после чего игрок может вводить ответ. Для таблицы Шульте вместо поля ввода отображается интерактивная сетка с ячейками, по которым нужно кликать в правильном порядке. Для заданий на эффект Струпа показывается слово, обозначающее цвет, в случайном цвете и поле для ввода ответа.

Все интерактивные элементы имеют обратную связь. Кнопка отправки ответа при наведении увеличивается в размерах. Ячейки таблицы Шульте при наведении подсвечиваются синим цветом, а при правильном клике становятся зелёными и блокируются для повторного нажатия. При неправильном клике ячейка ненадолго «трясётся» и подсвечивается красным. Это помогает игроку мгновенно понимать результат своих действий.

При ожидании подключения соперника отображается экран с анимированным индикатором загрузки – крутящимся кругом. Если игрок уже завершил прохождение всех заданий, но соперник ещё играет, появляется информационная подсказка, показывающая, сколько вопросов осталось ответить сопернику. Это исключает ситуацию, когда игрок думает, что игра зависла. На рисунке 5.4 показан экран ожидания с такой подсказкой.

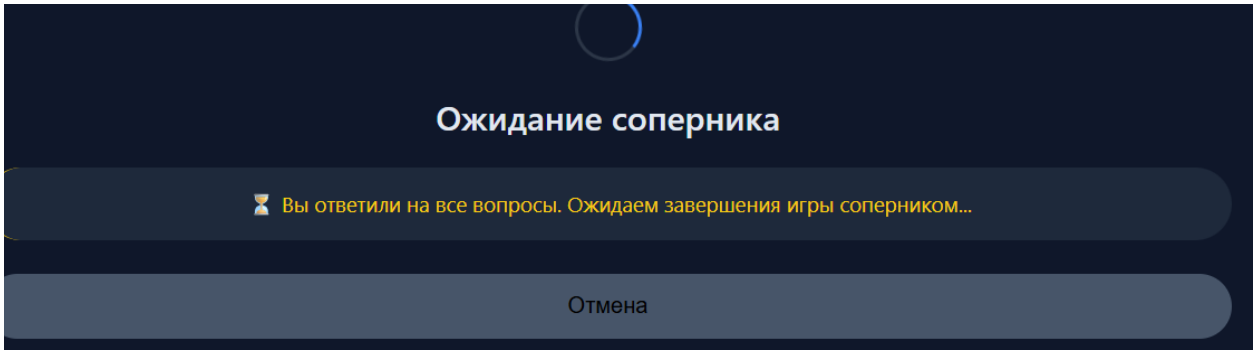


Рисунок 5.4 – Подсказка для игрока об ожидании ответов соперника

По окончании игры пользователь видит экран с результатом. При победе отображается трофей и сообщение «ПОБЕДА», при поражении – сообщение «ПОРАЖЕНИЕ», при ничьей – «НИЧЬЯ». Также выводится статистика правильных ответов обоих участников. На рисунке 5.5 показан экран победы.

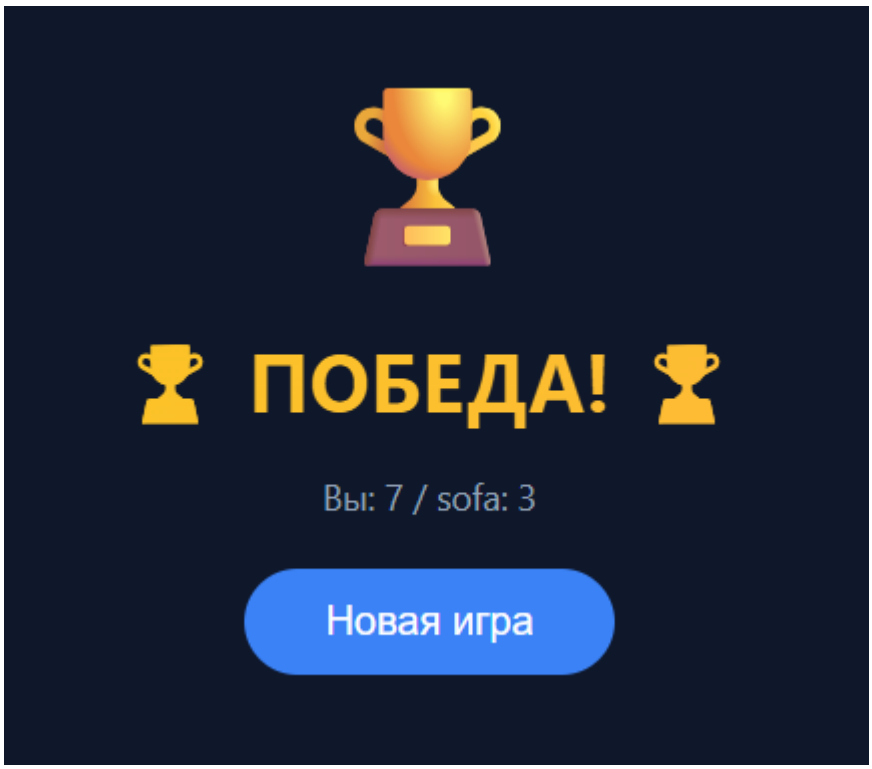


Рисунок 5.5 – Экран с сообщением о победе

В процессе игры пользователь получает всплывающие уведомления. Зелёное уведомление появляется при правильном ответе, красное – при неправильном или при истечении времени. Уведомления отображаются в центре экрана и автоматически исчезают через полторы секунды, не требуя дополнительных действий от игрока. При отключении соперника всплывает модальное окно с сообщением об ошибке и кнопкой OK, после нажатия на которую игра возвращается в начальное состояние (см.раздел 3.7).

# 6 ТЕСТИРОВАНИЕ ПРОГРАММЫ

## 6.1 Цели и методы тестирования

Целью тестирования разработанного приложения является проверка корректности работы всех функциональных компонентов, устойчивости сетевого взаимодействия и обработки ошибочных ситуаций.

В ходе тестирования использовались следующие методы:

– функциональное тестирование – проверка соответствия работы приложения заявленным требованиям;

– тестирование сетевых взаимодействий – анализ корректности установки и поддержания WebSocket-соединений;

– тестирование обработки ошибок – проверка реакции системы на нештатные ситуации.

## 6.2 Тестирование сетевого подключения

Тестирование сетевого взаимодействия включало проверку установки TCP-соединения, процедуры WebSocket-рукопожатия и обмена сообщениями между клиентом и сервером.

1 Проверка TCP-рукопожатия. При подключении клиента к серверу был проведён анализ сетевого трафика с помощью Wireshark. На рисунке 6.1 представлен захват пакетов, демонстрирующий успешное выполнение трёхстороннего рукопожатия. Видны три последовательных пакета: SYN от клиента, SYN-ACK от сервера и ACK от клиента, после чего соединение считается установленным. На рисунке виден порт 51085, который занимает клиент (Google Chrome) и порт 8080, который занимает сервер моего приложения.

| | | |
|---|---|---|
| TCP | 76 | 51085 → 8080 [SYN] Seq=0 Win=65535 Len=0 MSS=65475 WS=256 SACK_PERM |
| TCP | 76 | 8080 → 51085 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65475 WS=256 SACK_PERM |
| TCP | 64 | 51085 → 8080 [ACK] Seq=1 Ack=1 Win=65280 Len=0 |

Рисунок 6.1 – TCP-рукопожатие

2 Проверка WebSocket-рукопожатия. После установки TCP-соединения клиент отправляет HTTP-запрос с заголовками Upgrade. На рисунке 6.2 показан этот запрос, содержащий ключ Sec-WebSocket-Key. Сервер обрабатывает запрос и возвращает ответ с кодом 101 Switching Protocols и вычисленным ключом Sec-WebSocket-Accept, что подтверждает успешное переключение на протокол WebSocket. На рисунке 6.3 представлен ответ сервера.

```
532 17.313226    ::1    ::1    HTTP    615 GET / HTTP/1.1

Hypertext Transfer Protocol
> GET / HTTP/1.1\r\n
  Host: localhost:8081\r\n
  Connection: Upgrade\r\n
  Pragma: no-cache\r\n
  Cache-Control: no-cache\r\n
  User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.3
  Upgrade: websocket\r\n
  Origin: http://localhost:8080\r\n
  Sec-WebSocket-Version: 13\r\n
  Accept-Encoding: gzip, deflate, br, zstd\r\n
  Accept-Language: ru-BY,ru;q=0.9,be-BY;q=0.8,be;q=0.7,ru-RU;q=0.6,en-US;q=0.5,en;q=0.4\r\n
  Sec-WebSocket-Key: kxzLnNLnKZ7fRJSiHFBw+g==\r\n
  Sec-WebSocket-Extensions: permessage-deflate; client_max_window_bits\r\n
  \r\n
  [Response in frame: 534]
  [Full request URI: http://localhost:8081/]
```

Рисунок 6.2 – Запрос на переключение протоколов

```
534 17.314486    ::1    ::1    HTTP    193 HTTP/1.1 101 Switching Protocols

> Frame 534: Packet, 193 bytes on wire (1544 bits), 193 bytes captured (1544 bits) on interface \Device\NPF_Loopback, id 0
> Null/Loopback
> Internet Protocol Version 6, Src: ::1, Dst: ::1
> Transmission Control Protocol, Src Port: 8081, Dst Port: 50573, Seq: 1, Ack: 552, Len: 129
v Hypertext Transfer Protocol
  > HTTP/1.1 101 Switching Protocols\r\n
    Upgrade: websocket\r\n
    Connection: Upgrade\r\n
    Sec-WebSocket-Accept: k3zjOcIGwuAt+gC6XusQ63LT9fg=\r\n
    \r\n
    [Request in frame: 532]
    [Time since request: 1.260000 milliseconds]
    [Request URI: /]
    [Full request URI: http://localhost:8081/]
```

Рисунок 6.3 – Успешное переключение протоколов

3 Проверка обмена сообщениями. После установления соединения была проверена корректность передачи JSON-сообщений. Клиент отправил сообщение авторизации, на что сервер ответил подтверждением. На рисунке 6.4 показан фрагмент лога с успешной авторизацией пользователя (скриншот с консоли разработчика в браузере).

```
Получено:                                  websocket-client.js:22
{"type":"login_success","userId":4,"username":"vera","t
rophies":23}
```

Рисунок 6.4 – Сообщение об успешной авторизации

4 Проверка переподключения. При разрыве соединения (например, при закрытии вкладки браузера) сервер фиксирует событие закрытия сокета и удаляет пользователя из списка активных клиентов. При повторном подключении клиент выполняет авторизацию заново, и сервер корректно восстанавливает сессию. На рисунке 6.5 показан лог сервера при отключении клиента.

```
Ошибка соединения: Программа на вашем хост-компьютере разорвала установленное подключение
 Клиент 4 отключен от WebSocket
```

Рисунок 6.5 – Лог сервера (в IDEA) при отключении одного из игроков

5 Проверка отправки сообщений при отключённом клиенте. При попытке отправить сообщение пользователю, который отключился, сервер не генерирует исключение, а просто игнорирует отправку, о чём свидетельствует отсутствие ошибок в логах (рисунок 6.6 в консоли разработчика). Как видно на рисунке, первый лог о том, что соперник отключился, а второй лог о том, что приглашение не было отправлено этому пользователю.

```
Получено:                          websocket-client.js:22
{"type":"opponent_disconnected","message":"Соперник
отключился"}
Получено:                          websocket-client.js:22
{"type":"invite_error","error":"Пользователь не в
сети"}
```

Рисунок 6.6 – Отсутствие ошибок при отправке неактивному пользователю

### 6.3 Функциональное тестирование

Функциональное тестирование проводилось с использованием фреймворка JUnit 5, который позволяет создавать автоматизированные тесты для проверки отдельных модулей приложения [8]. Тестирование охватывает три основные области: работу с базой данных, игровую логику и сетевые взаимодействия.

1 Тестирование базы данных. В ходе тестирования были проверены операции регистрации нового пользователя, авторизации с корректными и некорректными учётными данными, а также предотвращение создания дублирующихся записей. Тесты подтвердили, что при попытке зарегистрировать пользователя с уже существующим логином система возвращает отрицательный результат, а при успешной регистрации в базе данных создаётся новая запись с корректными полями (логин, хеш пароля, аватар по умолчанию, нулевое количество трофеев). Успешное выполнение тестов можно увидеть на рисунке 6.7.

Рисунок 6.7 – Результат выполнения тестов

2 Тестирование игровой логики. Были проверены типы заданий. Для загадок подтверждено, что сравнение ответов выполняется без учёта регистра и игнорирует лишние пробелы. Для заданий «Запомни число» проверено точное совпадение вводимого числа с эталонным за определенное время. Для таблицы Шульте проверена корректная обработка правильной и неправильной последовательности нажатий. На рисунке 6.8 можно увидеть успешное прохождение всех вышеназванных тестов.

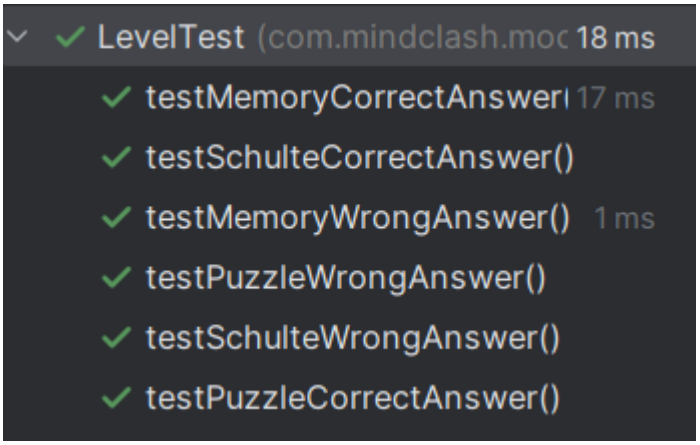


Рисунок 6.8 – Результат выполнения тестов

3 Тестирование сетевого взаимодействия. Для тестирования сетевого взаимодействия были разработаны тесты, проверяющие установку WebSocket-соединения, отправку и получение сообщений, а также обработку ошибок при неавторизованных запросах. Тесты используют библиотеку Java-WebSocket для создания клиентских подключений. На рисунке 6.9 можно увидеть успешное прохождение данных тестов.

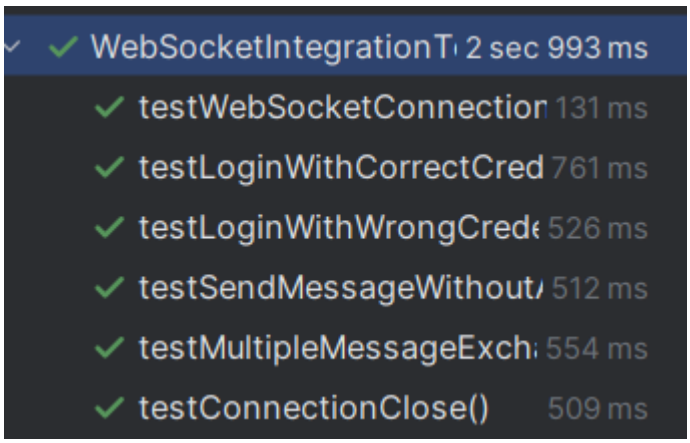


Рисунок 6.9 – Результат выполнения тестов

### 6.4 Анализ результатов тестирования

В ходе функционального и сетевого тестирования были проверены все основные компоненты разработанного приложения. Тестирование охватило три основные области: проверку модели данных, проверку операций с базой данных, проверку игровой логики и проверку сетевого взаимодействия.

Тестирование модели Level подтвердило корректную работу метода проверки ответов для заданий.

Тестирование операций с базой данных подтвердило корректную работу всех методов класса UserDAO. Регистрация нового пользователя успешно создаёт запись в базе данных с корректными значениями всех полей. При попытке зарегистрировать пользователя с уже существующим логином система возвращает отрицательный результат, предотвращая дублирование записей. Авторизация с верными учётными данными возвращает объект пользователя, а с неверными – нулевое значение. Методы обновления аватара и добавления трофеев корректно изменяют соответствующие поля в базе данных.

Тестирование сетевого взаимодействия подтвердило корректную работу WebSocket-сервера. Установка соединения происходит успешно, рукопожатие выполняется в соответствии со спецификацией протокола. Авторизация с верными учётными данными возвращает положительный ответ, а с неверными – сообщение об ошибке. При попытке отправить игровое сообщение без предварительной авторизации сервер возвращает сообщение об ошибке, что подтверждает корректную работу механизма безопасности. Также подтверждена корректная обработка закрытия соединения как со стороны клиента, так и со стороны сервера.

Таким образом, проведённое тестирование подтвердило корректность работы всех функциональных компонентов разработанного приложения. Система успешно обрабатывает регистрацию и авторизацию пользователей, проверяет ответы на задания всех типов, определяет победителя в различных игровых ситуациях и обеспечивает надёжное сетевое взаимодействие.

# 7 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ И АДМИНИСТРАТОРА

## 7.1 Руководство пользователя

1 Запуск и подключение к игре. Для начала работы необходимо открыть веб-браузер (Google Chrome, Mozilla Firefox или другой). В адресной строке браузера необходимо ввести адрес http://localhost:8080.

Игра с другом по сети.

2 Регистрация и вход в систему. При первом посещении страницы пользователь попадает на экран входа. Для регистрации необходимо нажать ссылку «Зарегистрироваться». На странице регистрации следует указать желаемое имя пользователя (логин), пароль и выбрать аватар из предложенных вариантов (кот, собака, робот, волшебник и т.д.). После заполнения всех полей необходимо нажать кнопку «Зарегистрироваться». При успешной регистрации пользователь будет перенаправлен на страницу входа. На странице входа необходимо ввести логин и пароль, затем нажать кнопку «Войти».

3 Главный экран. После успешного входа пользователь попадает на главный экран игры. В верхней части экрана отображается профиль пользователя с аватаром и именем. В центре экрана расположен выбор уровня сложности: лёгкий, средний или сложный. Ниже расположено поле для ввода ника друга, которому будет отправлено приглашение. Для начала игры необходимо ввести имя друга и нажать кнопку «Пригласить друга».

4 Получение приглашения. Приглашённый пользователь должен также быть авторизован в системе. При получении приглашения в правом нижнем углу экрана появляется всплывающее окно с сообщением «Приглашение» и именем пригласившего. Для принятия приглашения необходимо нажать кнопку «Принять», для отклонения – «Отклонить». После принятия приглашения игра начинается автоматически для обоих игроков.

5 Игровой процесс. После начала игры на экране отображается панель прогресса, показывающая информацию об обоих игроках: аватар, имя, количество правильных ответов и заполняющаяся полоса прогресса. В центральной части экрана показывается текущий номер уровня и круговой таймер, отсчитывающий время на ответ.

В зависимости от типа задания интерфейс меняется. Для заданий-загадок на экране отображается текст вопроса и поле для ввода ответа. Пользователь вводит ответ и нажимает кнопку отправки или клавишу Enter. При правильном ответе появляется зелёное уведомление, при неправильном – красное. Для заданий на запоминание числа на экране крупным шрифтом отображается число, которое через две секунды скрывается. Пользователь должен ввести запомненное число в поле ввода. Для таблицы Шульте на экране появляется квадратная сетка с числами, расположенными в случайном порядке. Пользователь должен последовательно нажимать на ячейки в

порядке возрастания чисел от 1 до N. При правильном нажатии ячейка подсвечивается зелёным цветом, при неправильном – красным.

На каждый ответ отводится ограниченное время, показывающееся в таймере. При истечении времени ответ автоматически считается неправильным, и игра переходит к следующему вопросу.

6 Окончание игры. Игра завершается, когда оба игрока ответили на все вопросы. Победителем становится игрок, набравший большее количество правильных ответов. При равном количестве правильных ответов объявляется ничья. По окончании игры отображается экран с результатом: при победе – трофей и сообщение «ПОБЕДА!», при поражении – сообщение «ПОРАЖЕНИЕ», при ничьей – сообщение «НИЧЬЯ!». Также выводится статистика правильных ответов обоих игроков. Для начала новой игры необходимо нажать кнопку «Новая игра».

7 Требования к браузеру. Для корректной работы игры рекомендуется использовать последние версии браузеров Google Chrome, Mozilla Firefox, Opera или Яндекс.Браузер. Не рекомендуется использовать Internet Explorer и старые версии Safari.

Игра в тренировочном режиме.

На стартовой странице приложения можно увидеть варианты игры в одиночном режиме. Для этого не нужно проходить авторизацию. Нажав на один из режимов игры, можно потренироваться в заданиях на эрудицию, запоминание, в виде таблицы Шульте, на эффект Струпа.

### 7.2 Руководство администратора

Назначение администратора: администратор системы отвечает за управление базой данных уровней, мониторинг работы сервера и обслуживание пользователей. В текущей версии приложения административные функции реализованы через прямое взаимодействие с базой данных MySQL, а также через мониторинг консоли сервера.

Функционал администратора. Администратор имеет возможность выполнять следующие действия.

1 Управление уровнями сложности. Администратор может добавлять новые вопросы, редактировать существующие и удалять устаревшие через SQL-запросы к базе данных. Каждый уровень содержит следующие поля: уникальный идентификатор, тип задания (загадка, запоминание числа, таблица Шульте, эффект Струпа), текст вопроса, правильный ответ, путь к изображению (при наличии), уровень сложности (1 – лёгкий, 2 – средний, 3 – сложный), время показа для заданий на запоминание числа (в секундах), размер сетки для таблицы Шульте (2, 3 или 4).

2 Просмотр статистики пользователей. Администратор может просматривать список зарегистрированных пользователей, их количество трофеев (побед), дату регистрации и выбранный аватар. Для этого необходимо выполнить SQL-запрос к таблице users.

3 Сброс данных пользователя. При необходимости администратор может сбросить количество трофеев пользователя или удалить учётную запись с помощью соответствующих SQL-запросов.

4 Мониторинг сервера. Администратор отслеживает логи работы сервера, которые выводятся в консоль. В логах отображаются все подключения и отключения клиентов, успешные и неудачные попытки авторизации, создание игровых комнат, ответы игроков и результаты игр. Пример вывода в консоли: «Клиент 4 подключен к WebSocket», «Игрок 4: ответ=ёлка, правильно=true, прогресс=3/10, очков=3», «Игра окончена! Победитель: vera».

5 Управление портами. Администратор может изменить порты для HTTP-сервера (по умолчанию 8080) и WebSocket-сервера (по умолчанию 8081).

6 Инструменты администратора. Для выполнения административных функций используются следующие инструменты: phpMyAdmin для работы с базой данных; консоль сервера для мониторинга событий в реальном времени; любой текстовый редактор для просмотра логов.

7 Действия при нештатных ситуациях. При возникновении ошибок администратор должен проверить логи сервера. Наиболее частые проблемы и их решения: «Address already in use» – порт занят другим приложением, необходимо остановить конфликтующую программу или изменить порт в настройках сервера; «Table 'mindclash.levels' doesn't exist» – отсутствуют таблицы в базе данных, необходимо выполнить SQL-скрипт инициализации; «Access denied for user» – неверные учётные данные для подключения к базе данных, необходимо проверить параметры подключения в файле DatabaseManager.java.

### 7.3 Настройка сервера и запуск

1 Требования к окружению. Для работы сервера необходимо установить следующее программное обеспечение: Java Development Kit (JDK) версии 21 или выше, MySQL Server версии 8.0 или выше, Maven для сборки проекта (можно использовать встроенные средства IntelliJ IDEA).

2 Настройка базы данных. Перед запуском сервера необходимо создать базу данных и таблицы. Для этого необходимо выполнить следующие шаги: запустить MySQL Server, подключиться к серверу MySQL с правами администратора. При первом запуске сервера таблицы создаются автоматически.

3 Настройка параметров подключения к базе данных. В файле DatabaseManager.java необходимо указать параметры подключения: URL базы данных, имя пользователя и пароль.

4 Настройка портов сервера. В файле MainServer.java можно изменить порты HTTP и WebSocket серверов. По умолчанию HTTP-сервер запускается на порту 8080, WebSocket-сервер – на порту 8081. При изменении портов

необходимо указать соответствующие номера. Если порт 8080 занят другим приложением, можно изменить его, например, на 8082.

5 Сборка проекта. Для сборки проекта необходимо выполнить команду mvn clean package в корневой директории проекта. В результате создается файл MindClash-1.0.0.jar в папке target.

6 В среде IntelliJ IDEA достаточно запустить класс MainServer с методом main.

7 Остановка сервера. Для корректной остановки сервера необходимо нажать сочетание клавиш Ctrl+C в консоли или остановить выполнение программы в среде разработки.

# ЗАКЛЮЧЕНИЕ

По результатам разработки программы «Веб-приложение, реализующее сетевую игру-головоломку «Mind Clash»», представляющее собой многопользовательскую игру-головоломку для двух игроков, соревнующихся в режиме реального времени, либо же игру для тренировки в одиночку.

Реализована клиент-серверная архитектура с использованием двух протоколов: HTTP для передачи статических файлов и обработки авторизации, и WebSocket для обеспечения двунаправленного обмена данными в реальном времени.

Разработана система управления пользователями, включающая регистрацию, авторизацию, хранение персональных данных (аватар, количество побед) в базе данных MySQL.

Реализована игровая логика, включающая 4 типа заданий различной сложности: текстовые загадки, задания на запоминание чисел, таблицы Шульте, задания на эффект Струпа. Каждый тип задания имеет свои особенности отображения и проверки, а также ограничение по времени выполнения.

Разработан графический интерфейс, адаптированный для различных разрешений экрана.

Среди недостатков программы можно выделить следующие: ограниченность набора заданий, отсутствие вариативности ответов для текстовых заданий, отсутствие перевода приложения на другие языки.

Программа обладает рядом преимуществ, таких как высокая скорость действий, возможность выполнения интерактивных заданий различных уровней сложности, возможность играть с другом по сети.

Дальнейшее развитие программы может включать в себя расширение функционала, добавление новых типов заданий, увеличение числа игроков, перевод игры на различные языки.

# СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Доманов, А.Т. Стандарт предприятия СТП 01-2024 / А. Т. Доманов, Н. И. Сорока. – Минск : БГУИР, 2024. – 178 с.

[2] ГОСТ 19.701-90. Схемы алгоритмов, программ, данных и систем.

[3] Россияне из всех игр предпочитают головоломки [Электронный ресурс]. – Режим доступа : https://rg.ru/2023/03/31/v-vk-sozdali-portret-tipichnogo-rossijskogo-gejmera.html/. Дата доступа: 11.03.2026.

[4] Лучшие онлайн-головоломки для игры с друзьями [Электронный ресурс]. – Режим доступа : https://fogplay.mts.ru/blog/luchshie-onlajn-golovolomki-dlya-igry-s-druzyami/. Дата доступа: 11.03.2026.

[5] Алгоритмы шифрования, на которых держится мир – Код [Электронный ресурс]. – Режим доступа : https://thecode.media/5-encrypts/. Дата доступа: 11.03.2026.

[6] WebSockets vs HTTP: Key Differences Explained [Электронный ресурс]. – Режим доступа : https://blog.postman.com/websockets-vs-http-key-differences-explained/. Дата доступа: 12.03.2026.

[7] WebSockets [Электронный ресурс]. – Режим доступа : https://learn.javascript.ru/websocket. Дата доступа: 22.03.2026.

[8] JUnit: основы тестирования в Java [Электронный ресурс]. – Режим доступа : https://skillbox.ru/media/code/junit-modulnoe-testirovanie-v-java-i-testdriven-development. Дата доступа: 17.04.2026.

## Приложение А (обязательное): Листинг кода с комментариями

Листинг кода файла HttpServer.java:

```
package com.mindclash.server;
import com.mindclash.database.UserDAO;
import com.mindclash.model.User;
import com.mindclash.utils.FileUtils;
import com.mindclash.utils.HttpResponseBuilder;
import java.io.*;
import java.net.ServerSocket;
import java.net.Socket;
import java.net.URLDecoder;
import java.util.HashMap;
import java.util.Map;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
// HTTP сервер для обработки статических файлов и авторизации
public class HttpServer {
    private static final int PORT = 8080;
    private ServerSocket serverSocket;
    private boolean running;
    private ExecutorService threadPool;
    // Запуск HTTP сервера
    public void start() {
        try {
            serverSocket = new ServerSocket(PORT);
            running = true;
            threadPool = Executors.newFixedThreadPool(10);
            System.out.println("   в браузере: http://localhost:" + PORT);
            while (running) {
                Socket clientSocket = serverSocket.accept();
                threadPool.execute(new ClientHandler(clientSocket));
            }
        } catch (IOException e) {
            System.err.println("Ошибка HTTP сервера: " + e.getMessage());
        }
    }
    // Остановка сервера
    public void stop() {
        running = false;
        threadPool.shutdown();
        try {
            if (serverSocket != null && !serverSocket.isClosed()) serverSocket.close();
        } catch (IOException e) { e.printStackTrace(); }
```

```
    }

    // Обработчик клиентского подключения
    private static class ClientHandler implements Runnable {
        private Socket clientSocket;
        public ClientHandler(Socket socket) { this.clientSocket = socket; }
        @Override
        public void run() {
            try (InputStream in = clientSocket.getInputStream();
                 OutputStream out = clientSocket.getOutputStream();
                 BufferedReader reader = new BufferedReader(new
InputStreamReader(in))) {
                String requestLine = reader.readLine();
                if (requestLine == null || requestLine.isEmpty()) return;
                String response = handleRequest(requestLine, reader);
                out.write(response.getBytes());
                out.flush();
            } catch (IOException e) {
                System.err.println("Ошибка обработки клиента: " + e.getMessage());
            } finally {
                try { clientSocket.close(); } catch (IOException e) {}
            }
        }
        // Обработка HTTP запроса
        private String handleRequest(String requestLine, BufferedReader reader)
throws IOException {
            String[] parts = requestLine.split(" ");
            if (parts.length < 2) return HttpResponseBuilder.badRequest("Неверный
формат запроса");
            String method = parts[0];
            String path = parts[1];
            System.out.println("   Запрос: " + method + " " + path);
            if (method.equals("POST")) return handlePost(path, reader);
            if (method.equals("GET")) return handleGet(path);
            return HttpResponseBuilder.badRequest("Метод не поддерживается");
        }
        // Обработка POST запроса
        private String handlePost(String path, BufferedReader reader) throws
IOException {
            String line;
            int contentLength = 0;
            // Чтение заголовков
            while (!(line = reader.readLine()).isEmpty()) {
                if (line.toLowerCase().startsWith("content-length:")) {
```

```
                contentLength = Integer.parseInt(line.substring(15).trim());
            }
        }
        // Чтение тела запроса
        char[] bodyChars = new char[contentLength];
        reader.read(bodyChars);
        String body = new String(bodyChars);
        System.out.println("   Тело POST: " + body);
        Map<String, String> params = parseUrlEncoded(body);
        if (path.equals("/register")) return handleRegister(params);
        if (path.equals("/login")) return handleLogin(params);
        return HttpResponseBuilder.notFound();
    }
    // Обработка GET запроса
    private String handleGet(String path) {
        String filePath;
        if (path.equals("/") || path.equals("/index.html")) filePath = "index.html";
        else if (path.equals("/websocket-test") || path.equals("/websocket-
test.html")) filePath = "websocket-test.html";
        else if (path.startsWith("/")) filePath = path.substring(1);
        else filePath = path;
        if (filePath.contains("?")) filePath = filePath.substring(0,
filePath.indexOf("?"));
        if (!FileUtils.fileExists(filePath)) return HttpResponseBuilder.notFound();
        String content = FileUtils.readWebFile(filePath);
        if (content == null) return HttpResponseBuilder.notFound();
        String mimeType = FileUtils.getMimeType(filePath);
        return new HttpResponseBuilder().setBody(content, mimeType).build();
    }
    // Парсинг данных из формы
    private Map<String, String> parseUrlEncoded(String body) {
        Map<String, String> params = new HashMap<>();
        if (body == null || body.isEmpty()) return params;
        String[] pairs = body.split("&");
        for (String pair : pairs) {
            String[] keyValue = pair.split("=");
            if (keyValue.length == 2) {
                try {
                    String key = URLDecoder.decode(keyValue[0], "UTF-8");
                    String value = URLDecoder.decode(keyValue[1], "UTF-8");
                    params.put(key, value);
                } catch (Exception e) { System.err.println("Ошибка декодирования: "
+ e.getMessage()); }
            }
```

```
        }
        return params;
    }
    // Обработка регистрации нового пользователя
    private String handleRegister(Map<String, String> params) {
        String username = params.get("username");
        String password = params.get("password");
        String avatar = params.get("avatar");
        if (username == null || password == null) return
HttpResponseBuilder.badRequest("Логин и пароль обязательны");
        boolean success = UserDAO.registerUser(username, password, avatar);
        if (success) {
            String html =
"<html><body><script>window.location='/login.html';</script></body></html>";
            return new HttpResponseBuilder().setHtmlBody(html).build();
        } else {
            String html = "<html><body><script>alert('Ошибка регистрации:
логин уже занят');window.location='/register.html';</script></body></html>";
            return new HttpResponseBuilder().setHtmlBody(html).build();
        }
    }
    // Обработка авторизации пользователя
    private String handleLogin(Map<String, String> params) {
        String username = params.get("username");
        String password = params.get("password");
        if (username == null || password == null) return
HttpResponseBuilder.badRequest("Логин и пароль обязательны");
        User user = UserDAO.loginUser(username, password);
        if (user != null) {
            String html = "<html><body><script>" +
                    "localStorage.setItem('userId', '" + user.getId() + "');" +
                    "localStorage.setItem('username', '" + user.getUsername() + "');" +
                    "localStorage.setItem('password', '" + password + "');" +
                    "localStorage.setItem('avatar', '" + user.getAvatar() + "');" +
                    "localStorage.setItem('trophies', '" + user.getTrophies() + "');" +
                    "window.location='/game.html';</script></body></html>";
            return new HttpResponseBuilder().setHtmlBody(html).build();
        } else {
            String html = "<html><body><script>alert('Неверный логин или
пароль');window.location='/login.html';</script></body></html>";
            return new HttpResponseBuilder().setHtmlBody(html).build();
        }
    }
}
```

```
    public static void main(String[] args) {
        HttpServer server = new HttpServer();
        server.start();
    }
}
```

Листинг кода файла WebSocketServer.java:

```
package com.mindclash.server;
import java.io.IOException;
import java.net.ServerSocket;
import java.net.Socket;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import com.mindclash.database.UserDAO;
import com.mindclash.model.Level;
import java.util.List;
// WebSocket сервер для обработки игровых сообщений
public class WebSocketServer {
    private int port;
    private ServerSocket serverSocket;
    private boolean running;
    private ExecutorService threadPool;
    private Map<Integer, WebSocketHandler> connectedClients;
    private Map<String, GameRoom> activeRooms;
    private Map<Integer, PendingInvite> pendingInvites;
    public WebSocketServer(int port) {
        this.port = port;
        this.connectedClients = new ConcurrentHashMap<>();
        this.activeRooms = new ConcurrentHashMap<>();
        this.pendingInvites = new ConcurrentHashMap<>();
        this.threadPool = Executors.newFixedThreadPool(20);
    }
    // Запуск WebSocket сервера
    public void start() {
        try {
            serverSocket = new ServerSocket(port);
            running = true;
            System.out.println(" WebSocket сервер запущен на порту " + port);
            while (running) {
                Socket clientSocket = serverSocket.accept();
                WebSocketHandler handler = new WebSocketHandler(clientSocket, this);
                threadPool.execute(handler);
```

```
            }
        } catch (IOException e) {
            System.out.println("Ошибка WebSocket сервера: " + e.getMessage());
        }
    }
    // Остановка сервера
    public void stop() {
        running = false;
        threadPool.shutdown();
        try {
            if (serverSocket != null && !serverSocket.isClosed()) serverSocket.close();
        } catch (IOException e) { e.printStackTrace(); }
    }
    // Регистрация клиента после успешной авторизации
    public void registerClient(int userId, WebSocketHandler handler) {
        connectedClients.put(userId, handler);
        System.out.println("Клиент " + userId + " подключен к WebSocket");
    }
    // Удаление клиента при отключении
    public void removeClient(int userId) {
        connectedClients.remove(userId);
        System.out.println("Клиент " + userId + " отключен от WebSocket");
        for (GameRoom room : activeRooms.values()) {
            if (room.hasPlayer(userId)) {
                int opponentId = room.getOpponentId(userId);
                room.setPlayerDisconnected(userId);
                if (opponentId != -1 && connectedClients.containsKey(opponentId)) {
                    sendToUser(opponentId,
"{\"type\":\"opponent_disconnected\",\"message\":\"Соперник отключился\"}");
                }
                break;
            }
        }
    }
    // Отправка сообщения конкретному пользователю
    public void sendToUser(int userId, String message) {
        WebSocketHandler handler = connectedClients.get(userId);
        if (handler != null) handler.sendMessage(message);
    }
    // Рассылка сообщения всем в комнате (кроме указанного)
    public void broadcastToRoom(String roomId, String message, int excludeUserId)
{
        GameRoom room = activeRooms.get(roomId);
        if (room == null) return;
```

```
        for (int userId : room.getAllPlayers()) {
            if (userId != excludeUserId && connectedClients.containsKey(userId)) {
                sendToUser(userId, message);
            }
        }
    }
    // Создание новой игровой комнаты
    public String createRoom(int player1Id, String player1Name, int difficulty) {
        String roomId = generateRoomId();
        GameRoom room = new GameRoom(roomId, player1Id, player1Name,
difficulty);
        activeRooms.put(roomId, room);
        return roomId;
    }
    // Подключение второго игрока к комнате
    public boolean joinRoom(String roomId, int player2Id, String player2Name) {
        GameRoom room = activeRooms.get(roomId);
        if (room == null) return false;
        if (room.getPlayer2Id() != -1) return false;
        room.setPlayer2(player2Id, player2Name);
        return true;
    }
    // Получение комнаты по ID
    public GameRoom getRoom(String roomId) { return activeRooms.get(roomId);
}
    // Получение комнаты, в которой находится игрок
    public GameRoom getPlayerRoom(int userId) {
        for (GameRoom room : activeRooms.values()) {
            if (room.hasPlayer(userId)) return room;
        }
        return null;
    }
    // Отправка приглашения другому игроку
    public boolean sendInvite(int fromUserId, String fromUsername, int toUserId, int
difficulty) {
        if (!connectedClients.containsKey(toUserId)) return false;
        String inviteId = generateInviteId();
        PendingInvite invite = new PendingInvite(inviteId, fromUserId,
fromUsername, toUserId, difficulty);
        pendingInvites.put(toUserId, invite);
        sendToUser(toUserId, "{\"type\":\"invite\",\"inviteId\":\"" + inviteId +
"\",\"fromUserId\":" + fromUserId + ",\"fromUsername\":\"" + fromUsername +
"\",\"difficulty\":" + difficulty + "}");
        return true;
```

```
    }
    // Принятие приглашения
    public boolean acceptInvite(int userId, String inviteId) {
        PendingInvite invite = pendingInvites.get(userId);
        if (invite == null || !invite.getInviteId().equals(inviteId)) {
            System.out.println("Ошибка: приглашение не найдено");
            return false;
        }
        String player1Name = invite.getFromUsername();
        String player2Name = UserDAO.getUserById(userId).getUsername();
        System.out.println("Принятие приглашения: " + player1Name + " + " +
player2Name);
        String roomId = createRoom(invite.getFromUserId(), player1Name,
invite.getDifficulty());
        boolean joined = joinRoom(roomId, userId, player2Name);
        if (!joined) return false;
        pendingInvites.remove(userId);
        GameRoom room = activeRooms.get(roomId);
        if (room == null) return false;
        // Отправляем game_start обоим игрокам
        WebSocketHandler handler1 = connectedClients.get(invite.getFromUserId());
        if (handler1 != null) handler1.sendGameStart(room, invite.getFromUserId());
        WebSocketHandler handler2 = connectedClients.get(userId);
        if (handler2 != null) handler2.sendGameStart(room, userId);
        return true;
    }
    // Отклонение приглашения
    public void rejectInvite(int userId) {
        PendingInvite invite = pendingInvites.remove(userId);
        if (invite != null) {
            sendToUser(invite.getFromUserId(),
"{\"type\":\"invite_rejected\",\"message\":\"Игрок отклонил приглашение\"}");
            System.out.println("Приглашение отклонено: userId=" + userId);
        }
    }
    // Обработка ответа игрока на вопрос
    public void handleAnswer(int userId, String roomId, String answer) {
        GameRoom room = activeRooms.get(roomId);
        if (room == null) {
            System.out.println("Ошибка: комната не найдена " + roomId);
            return;
        }
        boolean correct = room.submitAnswer(userId, answer);
```

```
        sendToUser(userId,
String.format("{\"type\":\"answer_result\",\"correct\":%b}", correct));
        String progressUpdate = String.format(
"{\"type\":\"progress_update\",\"player1Correct\":%d,\"player2Correct\":%d,\"play
er1Answered\":%d,\"player2Answered\":%d}",
                room.getPlayer1Correct(), room.getPlayer2Correct(),
                room.getPlayerAnswered(room.getPlayer1Id()),
                room.getPlayer2Id() != -1 ?
room.getPlayerAnswered(room.getPlayer2Id()) : 0
        );
        for (int playerId : room.getAllPlayers()) sendToUser(playerId,
progressUpdate);
        // Проверка окончания игры
        if (room.bothFinished()) {
            int winnerId = room.getWinner();
            if (winnerId == -1) {
                String tieMsg = "{\"type\":\"game_over\",\"winnerId\":-
1,\"winnerUsername\":\"Ничья\"}";
                for (int playerId : room.getAllPlayers()) sendToUser(playerId, tieMsg);
                System.out.println("Игра окончена! НИЧЬЯ!");
            } else {
                String winnerName = room.getPlayerName(winnerId);
                String gameOverMsg =
String.format("{\"type\":\"game_over\",\"winnerId\":%d,\"winnerUsername\":\"%s
\"}", winnerId, winnerName);
                for (int playerId : room.getAllPlayers()) sendToUser(playerId,
gameOverMsg);
                UserDAO.addTrophy(winnerId);
                System.out.println("Игра окончена! Победитель: " + winnerName);
            }
            activeRooms.remove(roomId);
        }
    }
    // Генерация уникального ID комнаты
    private String generateRoomId() {
        return "ROOM_" + System.currentTimeMillis() + "_" + (int)(Math.random() *
10000);
    }
    // Генерация уникального ID приглашения
    private String generateInviteId() {
        return "INV_" + System.currentTimeMillis() + "_" + (int)(Math.random() *
10000);
    }
```

```
    // Внутренний класс для хранения ожидающих приглашений
    private static class PendingInvite {
        private String inviteId;
        private int fromUserId;
        private String fromUsername;
        private int toUserId;
        private int difficulty;
        public PendingInvite(String inviteId, int fromUserId, String fromUsername, int
toUserId, int difficulty) {
            this.inviteId = inviteId;
            this.fromUserId = fromUserId;
            this.fromUsername = fromUsername;
            this.toUserId = toUserId;
            this.difficulty = difficulty;
        }
        public String getInviteId() { return inviteId; }
        public int getFromUserId() { return fromUserId; }
        public String getFromUsername() { return fromUsername; }
        public int getToUserId() { return toUserId; }
        public int getDifficulty() { return difficulty; }
    }
}
```

Листинг кода файла WebSocketHandler.java:

```
package com.mindclash.server;
import com.mindclash.database.UserDAO;
import com.mindclash.model.User;
import com.mindclash.utils.WebSocketFrame;
import java.io.*;
import java.net.Socket;
// Обработчик WebSocket соединения для одного клиента
public class WebSocketHandler implements Runnable {
    private Socket socket;
    private WebSocketServer server;
    private InputStream in;
    private OutputStream out;
    private Integer userId;
    private boolean handshakeDone;
    private boolean running;
    public WebSocketHandler(Socket socket, WebSocketServer server) {
        this.socket = socket;
        this.server = server;
        this.handshakeDone = false;
        this.running = true;
```

```
}
@Override
public void run() {
    try {
        in = socket.getInputStream();
        out = socket.getOutputStream();
        while (running) {
            if (!handshakeDone) {
                if (!performHandshake()) break;
                handshakeDone = true;
            } else {
                String message = WebSocketFrame.decodeTextFrame(in);
                if (message == null) break;
                handleMessage(message);
            }
        }
    } catch (IOException e) {
        System.out.println("Ошибка соединения: " + e.getMessage());
    } finally {
        close();
    }
}
// Выполнение WebSocket handshake (переключение с HTTP на WebSocket)
private boolean performHandshake() throws IOException {
    BufferedReader reader = new BufferedReader(new InputStreamReader(in));
    String line;
    String websocketKey = null;
    line = reader.readLine();
    if (line == null || line.isEmpty()) return false;
    while (!(line = reader.readLine()).isEmpty()) {
        if (line.startsWith("Sec-WebSocket-Key:")) {
            websocketKey = line.substring(18).trim();
        }
    }
    if (websocketKey == null) return false;
    String acceptKey = WebSocketFrame.generateAcceptKey(websocketKey);
    PrintWriter writer = new PrintWriter(out);
    writer.print("HTTP/1.1 101 Switching Protocols\r\n");
    writer.print("Upgrade: websocket\r\n");
    writer.print("Connection: Upgrade\r\n");
    writer.print("Sec-WebSocket-Accept: " + acceptKey + "\r\n\r\n");
    writer.flush();
    return true;
}
```

```
    // Маршрутизация входящих сообщений по типу
    private void handleMessage(String message) {
        System.out.println("Получено: " + message);
        if (message.contains("\"type\":\"login\"")) handleLogin(message);
        else if (message.contains("\"type\":\"invite\"")) handleInvite(message);
        else if (message.contains("\"type\":\"accept_invite\"")) handleAcceptInvite(message);
        else if (message.contains("\"type\":\"reject_invite\"")) handleRejectInvite(message);
        else if (message.contains("\"type\":\"answer\"")) handleAnswer(message);
        else if (message.contains("\"type\":\"ping\"")) sendMessage("{\"type\":\"pong\"}");
        else if (message.contains("\"type\":\"update_profile\"")) handleUpdateProfile(message);
        else if (message.contains("\"type\":\"create_room\"")) handleCreateRoom(message);
        else if (message.contains("\"type\":\"join_room\"")) handleJoinRoom(message);
        else sendMessage("{\"type\":\"error\",\"error\":\"Неизвестный тип сообщения\"}");
    }
    // Авторизация пользователя
    private void handleLogin(String message) {
        try {
            String username = extractValue(message, "username");
            String password = extractValue(message, "password");
            System.out.println("WebSocket логин: " + username);
            User user = UserDAO.loginUser(username, password);
            if (user != null) {
                this.userId = user.getId();
                server.registerClient(userId, this);
                sendMessage("{\"type\":\"login_success\",\"userId\":" + userId + ",\"username\":\"" + username + "\",\"trophies\":" + user.getTrophies() + "}");
                System.out.println("WebSocket авторизация успешна: " + username);
            } else {
                sendMessage("{\"type\":\"login_error\",\"error\":\"Неверный логин или пароль\"}");
                System.out.println("WebSocket ошибка авторизации: " + username);
            }
        } catch (Exception e) {
            System.out.println("Ошибка авторизации: " + e.getMessage());
            sendMessage("{\"type\":\"error\",\"error\":\"Ошибка авторизации\"}");
        }
```

```
    }
    // Отправка приглашения другому игроку
    private void handleInvite(String message) {
        if (userId == null) {
            sendMessage("{\"type\":\"error\",\"error\":\"Не авторизован\"}");
            return;
        }
        String toUsername = extractValue(message, "toUsername");
        int difficulty = Integer.parseInt(extractValue(message, "difficulty"));
        User toUser = UserDAO.getUserByUsername(toUsername);
        if (toUser == null) {
            sendMessage("{\"type\":\"invite_error\",\"error\":\"Пользователь не
найден\"}");
            return;
        }
        boolean success = server.sendInvite(userId,
UserDAO.getUserById(userId).getUsername(), toUser.getId(), difficulty);
        if (success) sendMessage("{\"type\":\"invite_sent\",\"toUsername\":\"" +
toUsername + "\"}");
        else sendMessage("{\"type\":\"invite_error\",\"error\":\"Пользователь не в
сети\"}");
    }
    // Обновление профиля (пароль или аватар)
    private void handleUpdateProfile(String message) {
        if (userId == null) {
            sendMessage("{\"type\":\"profile_updated\",\"success\":false,\"error\":\"Не
авторизован\"}");
            return;
        }
        String password = extractValue(message, "password");
        String avatar = extractValue(message, "avatar");
        System.out.println("Обновление профиля для userId=" + userId);
        if (password != null && !password.isEmpty()) {
            UserDAO.updatePassword(userId, password);
          sendMessage("{\"type\":\"profile_updated\",\"success\":true,\"password\":\""
+ password + "\"}");
        } else if (avatar != null && !avatar.isEmpty()) {
            UserDAO.updateAvatar(userId, avatar);
            sendMessage("{\"type\":\"profile_updated\",\"success\":true,\"avatar\":\"" +
avatar + "\"}");
        } else {

sendMessage("{\"type\":\"profile_updated\",\"success\":false,\"error\":\"Нет
данных для обновления\"}");
```

```
        }
    }
    // Принятие приглашения
    private void handleAcceptInvite(String message) {
        if (userId == null) return;
        String inviteId = extractValue(message, "inviteId");
        server.acceptInvite(userId, inviteId);
    }
    // Отклонение приглашения
    private void handleRejectInvite(String message) {
        if (userId == null) return;
        server.rejectInvite(userId);
    }
    // Обработка ответа на вопрос
    private void handleAnswer(String message) {
        if (userId == null) {
            sendMessage("{\"type\":\"error\",\"error\":\"Не авторизован\"}");
            return;
        }
        String roomId = extractValue(message, "roomId");
        String answer = extractValue(message, "answer");
        if (roomId == null) {
            sendMessage("{\"type\":\"error\",\"error\":\"Неверный формат ответа: нет
roomId\"}");
            return;
        }
        System.out.println("Обработка ответа от пользователя " + userId + " в
комнате " + roomId + ": " + answer);
        server.handleAnswer(userId, roomId, answer);
    }
    // Создание комнаты (устаревший метод, используется invite)
    private void handleCreateRoom(String message) {
        if (userId == null) return;
        int difficulty = Integer.parseInt(extractValue(message, "difficulty"));
        String roomId = server.createRoom(userId,
UserDAO.getUserById(userId).getUsername(), difficulty);
        sendMessage("{\"type\":\"room_created\",\"roomId\":\"" + roomId + "\"}");
    }
    // Присоединение к комнате по ID (устаревший метод, используется invite)
    private void handleJoinRoom(String message) {
        if (userId == null) return;
        String roomId = extractValue(message, "roomId");
        boolean success = server.joinRoom(roomId, userId,
UserDAO.getUserById(userId).getUsername());
```

```
        if (success) sendMessage("{\"type\":\"room_joined\",\"roomId\":\"" + roomId
+ "\"}");
        else sendMessage("{\"type\":\"error\",\"error\":\"Не удалось присоединиться
к комнате\"}");
    }
    // Отправка сообщения о начале игры с полными данными уровней
    public void sendGameStart(GameRoom room, int targetUserId) {
        StringBuilder sb = new StringBuilder();
        sb.append("{");
        sb.append("\"type\":\"game_start\",");
          sb.append("\"roomId\":\"").append(escapeJson(room.getRoomId())).append(
"\",");
        sb.append("\"totalLevels\":").append(room.getLevelsCount()).append(",");
        sb.append("\"player1Id\":").append(room.getPlayer1Id()).append(",");
        sb.append("\"player2Id\":").append(room.getPlayer2Id()).append(",");
        sb.append("\"levels\":").append(room.getLevelsJsonString());
        int opponentId = room.getOpponentId(targetUserId);
        if (opponentId != -1 && room.getPlayerName(opponentId) != null) {
sb.append(",\"opponentUsername\":\"").append(escapeJson(room.getPlayerName(
opponentId))).append("\"");
        }
        sb.append("}");
        sendMessage(sb.toString());
        System.out.println("Отправлен game_start пользователю " + targetUserId);
    }
    // Экранирование специальных символов для JSON
    private String escapeJson(String s) {
        if (s == null) return "";
        return s.replace("\\", "\\\\")
                .replace("\"", "\\\"")
                .replace("\n", "\\n")
                .replace("\r", "\\r")
                .replace("\t", "\\t");
    }
    // Извлечение значения поля из JSON строки
    private String extractValue(String json, String key) {
        String searchKey = "\"" + key + "\":\"";
        int start = json.indexOf(searchKey);
        if (start == -1) {
            searchKey = "\"" + key + "\":";
            start = json.indexOf(searchKey);
            if (start == -1) return null;
            start += searchKey.length();
            int end = json.indexOf(",", start);
```

```
            if (end == -1) end = json.indexOf("}", start);
            String value = json.substring(start, end).trim();
            return value.replace("\"", "");
        }
        start += searchKey.length();
        int end = json.indexOf("\"", start);
        if (end == -1) return null;
        return json.substring(start, end);
    }
    // Отправка сообщения клиенту
    public void sendMessage(String message) {
        try {
            byte[] frame = WebSocketFrame.encodeTextFrame(message);
            if (frame != null && frame.length > 0) {
                out.write(frame);
                out.flush();
            }
        } catch (IOException e) {
            System.out.println("Ошибка отправки: " + e.getMessage());
        }
    }
    // Закрытие соединения
    private void close() {
        running = false;
        if (userId != null) server.removeClient(userId);
        try { socket.close(); } catch (IOException e) { e.printStackTrace(); }
    }
}
```